



Final Year Project

NexFit

Project Team

Muhammad Sarim Khan 2012355

Muhammad Izhan Atif 2012353

Project Advisor

Usama Khalid

Submitted in partial fulfillment of the requirement for the degree
Bachelors of Science in Computer Science

In the

Faculty of Computing and Engineering Sciences
Shaheed Zulifkar Ali Bhutto Institute of Science and Technology
(SZABIST)

Karachi

Declaration Of Authorship

We, Muhammad Sarim Khan (2012355) and Muhammad Izhan Atif (2012353), declare that this report titled, “NexFit” and the work presented in it are our own. We confirm that:

This work was done wholly or mainly while in candidature for a bachelor’s degree at this University.

Where any part of this report has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.

Where we have consulted the published work of others, this is always clearly attributed.

Where we have quoted from the work of others, the source is always given. With the exception of such quotations, this report is entirely our own work.

We have acknowledged all main sources of help.

Where the report is based on work done by us jointly with others, we have made clear exactly what was done by others and what we have contributed ourselves.

Signed:

Muhammad Sarim Khan (2012355)

Muhammad Izhan (2012353)

Acknowledgement

In the name of ALLAH the most beneficent and merciful who gave us the knowledge and courage to work on this research area.

The success and final outcome of this project required a lot of guidance and assistance from many people and we are extremely privileged to have got this all along the completion of our project.

We would first like to thank our supervisor Mr. Usama Khalid of the Computer Science faculty at Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology. The door to his office was always open whenever we ran into a trouble spot or had a question about our research or writing. He consistently helped, cooperated and motivated us throughout the research.

We would like to thank our teachers who guided us in the light of their knowledge and experience. We would also like to express our gratitude to our loving parents and family members who helped and gave us encouragement. Furthermore, we would also like to acknowledge with much appreciation the crucial role of the staff of SZABIST, who gave the permission to use all required equipment and the necessary materials to complete the project.

At the end, we would like to thank Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology for providing us with such an inspiring environment. The quality education, the cooperative faculty members and the challenging environment have always motivated and boosted the confidence level of each and every student who has been a part of Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology.

Table of Contents

Project Proposal.....	1
1. Introduction.....	1
2. Objective.....	1
3. Problem Description.....	1
What?.....	1
Why?.....	1
4. Methodology.....	2
5. Project Scope.....	3
Assumptions:.....	3
6. Feasibility Study.....	3
Risks Involved:.....	3
Resource Requirements:.....	3
7. Solution Application Areas.....	4
8. Tools/Technology.....	4
1. Hardware:.....	4
2. Software:.....	4
3. Mobile or Web Development Tools:.....	4
4. Testing and Deployment Tools:.....	4
9. Expertise of the Team Members.....	5
10. Milestones.....	5
11. Project Schedule.....	7
12. Work Breakdown Structure.....	8
Fall Semester.....	8
Week 1 - 2.....	8
Week 3 - 7.....	8
Week 8 - 18.....	8
Spring Semester.....	8
Week 19 - 25.....	8
Week 26 - 32.....	8
Week 33-36.....	8
Week 37-38:.....	8
System Requirements Document.....	9
1. Introduction.....	9
1.1 Purpose.....	9
1.2 Document Conventions.....	9
1.3 Intended Audience and Reading Suggestions.....	9
1.4 Product Scope.....	9
1.5 References.....	9
2. Overall Description.....	10

2.1 Product Perspective.....	10
2.2 Product Functions.....	10
2.3 User Classes and Characteristics.....	11
2.4 Operating Environment.....	11
2.5 Design and Implementation Constraints.....	12
2.6 User Documentation.....	12
2.7 Assumptions and Dependencies.....	12
3. External Interface Requirements.....	12
3.1 User Interfaces.....	12
3.2 Hardware Interfaces.....	12
3.3 Software Interfaces.....	13
3.4 Communications Interfaces.....	13
4. System Features.....	13
5. Other Nonfunctional Requirements.....	17
5.1 Performance Requirements.....	17
5.2 Safety Requirements.....	17
5.3 Security Requirements.....	17
5.4 Software Quality Attributes.....	18
5.5 Business Rules.....	18
6. Other Requirements.....	18
6.1 System Sequence Diagrams.....	19
Software Design Specification.....	23
1. Introduction.....	23
1.1 Purpose of this document.....	23
1.2 Scope of the development project.....	23
1.3 Definitions, acronyms, and abbreviations.....	23
1.4 References.....	23
1.5 Overview of document.....	23
2. System architecture description.....	24
2.1. Section Overview.....	24
2.2. General Constraints.....	24
2.3. Data Design.....	25
2.4. Program Structure.....	26
3. Detailed description of components.....	26
3.1. Section Overview.....	26
3.2. Component n Detail.....	27
3.2.1. User Registration.....	27
3.2.2. Login.....	27
3.2.3. Add Exercise.....	28
3.2.4. Add Tokens.....	28
3.2.5. Check Analytics.....	28

3.2.6. Add Workout.....	29
3.2.7. Add Diet Plan.....	29
3.2.8. Chat.....	29
3.2.9. Scan QR.....	30
4. User Interface Design.....	30
4.1. Section Overview.....	30
4.2. Interface Design Rules.....	30
4.3. GUI Components.....	31
4.4. Detailed Description.....	31
4.4.1. Admin Login.....	31
4.4.2. Dashboard.....	31
4.4.3. Add Member.....	32
4.4.4. Add Trainer.....	32
4.4.5. Members List.....	33
4.4.6. Trainers List.....	33
4.4.7. Add Equipment.....	34
4.4.8. Equipment List.....	34
4.4.9. Exercises List.....	35
4.4.10. Add Token Plan.....	35
4.4.11. Purchase Tokens.....	36
4.4.12. Mobile Application Home.....	37
4.4.13. Login.....	37
4.4.14. Member Home.....	38
4.4.15. Member Diet Plan.....	39
4.4.16. Member Chat.....	39
4.4.17. Member Attendance.....	40
4.4.18. Exercises.....	40
4.4.19. Member Profile.....	41
4.4.20. Trainer Home.....	41
4.4.21. Workout Plan.....	42
4.4.22. Diet Plan.....	44
4.4.23. Trainer Chat.....	45
4.4.24. Trainer Profile.....	46
5. Reuse and relationships to other products.....	46
6. Design decisions and tradeoffs.....	46
7. Appendices.....	47
7.1. Class Diagram.....	47
7.2. Object Diagram.....	48
7.3. State Chart Diagram.....	49
7.4. Activity Diagrams.....	50
7.4.1. User Registration.....	50

7.4.2. Login.....	51
7.4.3. Add Exercise.....	52
7.4.4. Add Tokens.....	53
7.4.5. Check Analytics.....	54
7.4.6. Add Workout.....	55
7.4.7. Add Diet Plan.....	56
7.4.8. Chat.....	57
7.4.9. Track Progress.....	58
7.4.10. Scan QR.....	59
7.5. Sequence Diagrams.....	60
7.5.1. User Registration.....	60
7.5.2. Login.....	60
7.5.3. Add Exercise.....	61
7.5.4. Add Tokens.....	61
7.5.5. Check Analytics.....	62
7.5.6. Add Workout.....	62
7.5.7. Add Diet Plan.....	62
7.5.8. Chat.....	63
7.5.9. Track Progress.....	63
7.6. Collaboration Diagrams.....	64
7.6.1. User Registration.....	64
7.6.2. Login.....	64
7.6.3. Add Exercise.....	64
7.6.4. Add Tokens.....	65
7.6.5. Check Analytics.....	65
7.6.6. Add Workout.....	65
7.6.7. Add Diet Plan.....	65
7.6.8. Chat.....	66
7.7. Use-Case Diagram.....	66
7.8. Component Diagram.....	67
7.9. State Block Diagram.....	67
7.10. Entity Relationship Diagram.....	68
7.11. Test Cases.....	69
7.11.1. User Registration.....	69
7.11.2. Login.....	70
7.11.3. Add Exercise.....	70
7.11.4. Add Tokens.....	71
7.11.5. Check Analytics.....	71
7.11.6. User Login.....	72
7.11.7. User Profile.....	72
7.11.8. Exercises.....	73

7.11.9. Scan QR.....	73
7.11.10. Chat.....	74
7.11.11. Diet Plan.....	74
7.11.12. Workout Plan.....	75

Project Proposal

1. Introduction

The Gym Management Application is a digital solution that transforms the fitness industry by introducing a token-based system for gym memberships. Members receive tokens credited to their accounts, which are deducted for each gym visit. The application incorporates QR scanning for accurate attendance tracking. It offers a comprehensive gym management system, including registration, trainer management, diet plans, exercise guidelines, weight tracking, and chat functionality. The application aims to optimize gym operations, increase member engagement, and create revenue opportunities through trainer partnerships. By digitizing memberships and enhancing the gym experience, the application provides flexibility for members and efficient tools for gym owners. It revolutionizes the fitness journey, improving convenience and personalization.

2. Objective

To develop a digital token-based Gym Management Application that optimizes gym operations, enhances member experience, and increases revenue opportunities through personalized fitness guidance and efficient management tools.

3. Problem Description

What?

The Gym Management Application is a revolutionary solution that transforms traditional gym memberships through a digital token system. Members receive tokens credited to their accounts, which are deducted for each gym visit. This system provides a fair and efficient way to track membership utilization and attendance.

Why?

Traditional monthly gym memberships often lead to frustration for both gym owners and members. Members struggle with paying fees when they are unable to attend the gym due to various reasons, resulting in a waste of their investment. Gym owners face revenue loss from underutilized memberships. The Gym Management Application addresses these challenges head-on by offering a solution that benefits both parties.

The application offers members flexible attendance options, personalized fitness guidance, and a comprehensive platform to track their progress. It provides access to diet plans, exercise guidelines, and a built-in chat system to connect with trainers. For gym owners, the application

offers efficient management tools, increased member engagement, and potential revenue opportunities through partnerships with trainers.

By adopting this innovative solution, gym owners can enhance member satisfaction, improve retention rates, and streamline operations. Members benefit from a tailored fitness experience and the freedom to manage their membership according to their individual needs.

The Gym Management Application revolutionizes the fitness industry by combining convenience, personalization, and efficient management. It creates a win-win scenario for gym owners and members, leading to a more engaged and satisfied fitness community.

4. Methodology

The development of the Gym Management Application follows a systematic and iterative approach to ensure a successful outcome. The methodology consists of the following key steps:

1. Requirement Analysis: Gather detailed requirements from gym owners, trainers, and potential users to understand their needs and expectations from the application.

2. Design and Prototyping: Create a user-friendly interface design and develop interactive prototypes to visualize the application's flow and functionality.

3. Technology Selection: Choose the appropriate technology stack for development, considering factors such as scalability, performance, and cross-platform compatibility.

4. Development and Testing: Implement the application's features, including the digital token system, QR scan, user account management, diet plans, exercise guidelines, and chat system. Conduct thorough testing to ensure functionality, usability, and security.

5. Integration and Deployment: Integrate the developed components into a unified system and deploy the application on web and mobile platforms.

6. User Acceptance Testing: Conduct extensive testing with gym owners, trainers, and users to gather feedback and make necessary improvements.

7. Training and Support: Provide comprehensive training to gym owners, trainers, and users to ensure a smooth transition to the new system. Offer ongoing technical support and maintenance to address any issues that may arise.

By following this methodology, we aim to deliver a robust and user-friendly Gym Management Application that meets the needs of both gym owners and members while ensuring a seamless and engaging fitness experience.

5. Project Scope

The Gym Management Application focuses on designing and developing a comprehensive web and mobile application for gym owners, trainers, and users. It includes features like digital token system, QR scanning, user account management, diet plans, exercise guidelines, and chat system. The scope does not cover hardware components or physical infrastructure for gyms.

Assumptions:

- The application will be developed as a standalone system.
- Reliable internet connectivity is assumed for user access.
- Compatible QR devices will be used for scanning.

By clearly defining the project scope and assumptions, we can concentrate on delivering a robust and user-friendly Gym Management Application without getting involved in irrelevant technical details or hardware aspects.

6. Feasibility Study

With the defined scope, we have identified the following aspects:

Risks Involved:

1. Technical challenges in implementing QR scanning.
2. User adoption and acceptance of the digital token system.
3. Integration issues with existing gym management systems.

To cater to these risks, we will conduct thorough research, provide user training, and collaborate with gym owners and IT teams.

Resource Requirements:

1. Computing Resources: Sufficient processing power and memory.
2. Development Tools: Frameworks, IDEs, and programming languages.
3. Database Management System: To store user information and membership details.
4. Testing Devices: QR scanning devices and compatible hardware.

By addressing the risks and ensuring the necessary resources, we aim to meet the project schedule and deliver the Gym Management Application successfully.

7. Solution Application Areas

Our project targets the fitness and wellness industry, offering a Gym Management Application that brings digital transformation to gym operations. It enhances member experiences, streamlines operations, increases engagement and retention, and provides valuable data-driven insights. The application holds real value for the industry, improving overall efficiency and contributing to the growth and success of fitness businesses.

8. Tools/Technology

The Gym Management Application requires the following Hardware/Software tools and technologies:

1. Hardware:

- Servers or cloud infrastructure for hosting the application and managing data storage.

2. Software:

- Programming languages such as JavaScript, HTML, and CSS for frontend development.
- Frameworks and libraries like React or Angular for building the user interface.
- Backend development using languages like Python, Node.js, or Java.
- Database management systems like MySQL or MongoDB for storing member data and exercise information.
- Chat system integration for enabling communication between users, trainers, and administrators.
- Data analytics and reporting tools for generating insights from member data.

3. Mobile or Web Development Tools:

- Development platforms like React Native or Flutter for building cross-platform mobile applications.
- Integrated Development Environments (IDEs) such as Visual Studio Code or Android Studio.

4. Testing and Deployment Tools:

- Automated testing frameworks for ensuring the application's functionality and performance.
- Version control systems like Git for managing code changes and collaboration.
- Continuous integration and deployment tools for automating the testing and deployment processes.

9. Expertise of the Team Members

Yes, both the team members are equipped with the relevant knowledge and have previous experience with the required frameworks.

10. Milestones

Based on the Gym Management Application project, the major tasks/sub-tasks and associated milestones can be outlined as follows:

1. Project Initiation

- Define project scope, objectives, and deliverables.
- Conduct stakeholder analysis and gather requirements.
- Create a context diagram and actor-goal matrix to identify project milestones.

2. System Design and Architecture

- Design the overall system architecture, including frontend and backend components.
- Develop the database schema and data models.
- Create wireframes and user interface designs for the application.

3. User Management and Authentication

- Implement user registration and login functionality.
- Develop user profile management features.

4. Membership Management

- Implement membership purchase and renewal functionality.
- Track and manage user tokens for gym access.

5. Gym Management Features

- Develop a comprehensive exercise library with instructions and images.
- Create a diet plan management system for users.
- Implement a chat system for communication between users, trainers, and admins.

6. Reporting and Analytics

- Develop data analytics features to track user activity and progress.

- Generate reports on membership statistics, attendance, and revenue.
- Provide insights and recommendations based on user data.

7. Testing and Quality Assurance

- Conduct thorough testing of all application features and functionalities.
- Identify and fix any bugs or issues.
- Ensure the application meets performance and security standards.

8. Deployment and Maintenance

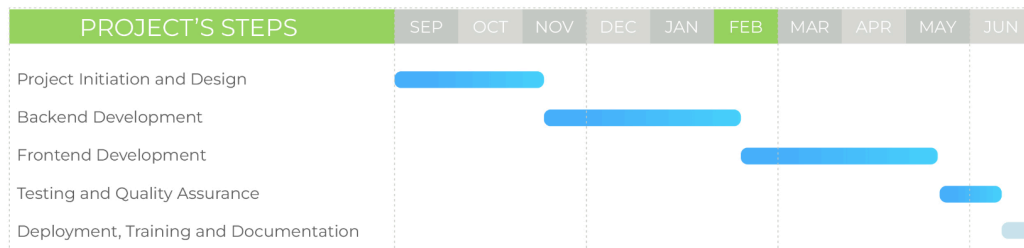
- Prepare the application for deployment on web and mobile platforms.
- Provide ongoing maintenance and support for the application.
- Continuously monitor and optimize the system for performance and scalability.

Please note that this is a general outline and the specific tasks may vary based on the project requirements and priorities.

11. Project Schedule

GANTT CHART

MONTHLY PROGRESS



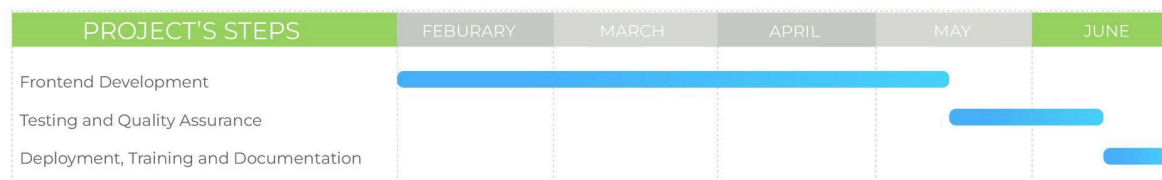
GANTT CHART

FALL SEMESTER



GANTT CHART

SPRING SEMESTER



12. Work Breakdown Structure

Fall Semester

Week 1 - 2

- Project Scope
- Requirement Gathering

Week 3 - 7

- Database Design (MySQL)
- UI/UX Design (Figma)

Week 8 - 18

- Backend Development (Python/Django Rest Framework)
 - Logic
 - API Development
 - Database Integration

Spring Semester

Week 19 - 25

- Frontend Development (WEB)
 - Web App UI Design
 - Web App Frontend Development

Week 26 - 32

- Frontend Development (MOBILE)
 - Mobile App UI Design
 - Mobile App Frontend Development

Week 33-36

- Testing and Quality Assurance
 - Unit Testing
 - Integration Testing
 - Quality Assurance

Week 37-38:

- Deployment
 - Mobile App Deployment
 - Web App Deployment
 - User Training (Member and Trainer)
 - Documentation

System Requirements Document

1. Introduction

1.1 Purpose

The Gym Management Application is a digital solution that transforms the fitness industry by introducing a token-based system for gym memberships. Members receive tokens credited to their accounts, which are deducted for each gym visit. The application incorporates QR scanning for accurate attendance tracking. It offers a comprehensive gym management system, including registration, trainer management, diet plans, exercise guidelines, weight tracking, and chat functionality. The application aims to optimize gym operations, increase member engagement, and create revenue opportunities through trainer partnerships. By digitizing memberships and enhancing the gym experience, the application provides flexibility for members and efficient tools for gym owners. It revolutionizes the fitness journey, improving convenience and personalization.

1.2 Document Conventions

We have used Times new roman font for headings with bold 14pt font size, 12pt bold times font for subheadings and arial font with 11pt font size for paragraphs.

1.3 Intended Audience and Reading Suggestions

- Gym owners/Administrators, who want to integrate NexFit into their organization.
- Gym goers, who are members of gyms that have NexFit integrated.
- Programmers who are interested in working on the project by developing it further or fixing bugs in it.

1.4 Product Scope

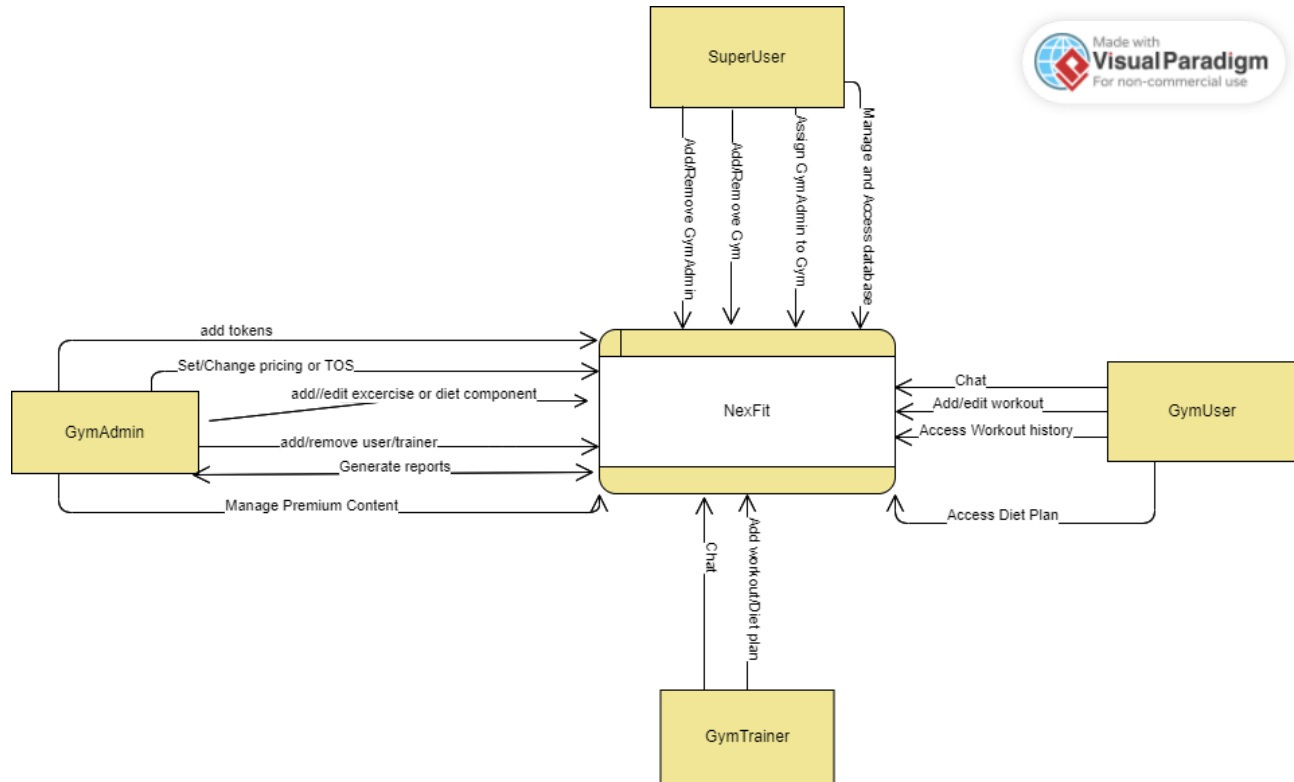
NexFit stands as a cutting-edge fitness management solution revolutionizing the fitness industry. Its core design features three distinct user roles: Superusers, GymAdmins, GymTrainers, and GymUsers. Superusers, equipped with the power of the Django admin panel, oversee the entire platform. GymAdmins, selected by Superusers, manage GymUsers efficiently through a dedicated React admin interface. GymUsers, the end clients, experience a personalized fitness journey via a feature-rich mobile application. The platform introduces a token system, ensuring a reliable attendance system that helps the user to save money. Beyond basic functionalities, NexFit offers tailored fitness plans, exercise routines, diet management, and real-time chat functionality for direct communication between trainers and users.

1.5 References

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

2. Overall Description

2.1 Product Perspective



2.2 Product Functions

User Management:

- Superuser: Manages GymAdmins, GymUsers, and platform policies.
- GymAdmin: Manages GymUsers associated with their gym.
- GymTrainer: Manages exercises and fitness plans for GymUser.
- GymUser: Accesses fitness plans, exercises, and interacts with GymTrainers.

Authentication and Authorization:

- User Login: Users can log in with their username and password.
- Role-Based Access Control: GymAdmins manage users, GymTrainers & GymUsers access fitness content.

Gym and Admin Management:

- Create Gyms: Superusers can add new gyms to the platform.
- Manage GymAdmins: Superusers assign GymAdmins to specific gyms.
- Admin Dashboard: GymAdmins have a dedicated panel to manage GymUsers and gym-specific activities.

Fitness Content:

- Access Plans: GymUsers can access fitness plans, exercises, and diet recommendations.
- Content Moderation: GymAdmin moderate and manage content to ensure quality and appropriateness.

Token System:

- Token Distribution: GymAdmin can add tokens to GymUser accounts according to their purchase.
- Token Usage: GymUsers use tokens to access the gym and services within the platform.

Reporting and Analytics:

- Generate Reports: GymAdmins can access reports and analytics for user engagement and financial transactions.
- Data Visualization: Graphical representation of data for better insights and decision-making.

Platform Customization:

- Content Management: GymAdmins manage and curate fitness plans, exercises, and diet plans available on the platform.
- Policy Configuration: GymAdmins set platform rules, pricing structures, and terms of service.

Communication:

- Messaging: GymTrainers and GymUsers can communicate within the platform for support and queries.

Security and Privacy:

- User Data Protection: Ensuring user data security and privacy in compliance with regulations.

Platform Support:

- Help Center: FAQs and support resources for users.

2.3 User Classes and Characteristics

- GymAdmin: GymAdmins will mostly be gym owners or receptionists etc. These people can be not tech savvy at all or have basic knowledge.
- GymTrainer: Trainers are people who are usually not very tech inclined but they mostly will have enough technical knowledge.
- GymUser: Users can be very technical or with no tech knowledge at all.

2.4 Operating Environment

The admin panel will operate on a web application that can be accessed through any browser. The main NexFit application is a mobile application for both iOS and android.

2.5 Design and Implementation Constraints

There were no design or implementation constraints.

2.6 User Documentation

There are two specification documents available for this application.

- 1 - SRS - Software Requirement specifications
- 2 - SDS - Software Design Specifications

2.7 Assumptions and Dependencies

We assume that all users and trainers have a relatively modern smartphone. We also assume that the gym will have a computer with an active internet connection.

3. External Interface Requirements

3.1 User Interfaces

W3 standards are used for the web application. Whereas android and iOS standards are used for the mobile application. Some mockups for the mobile application are seen below.



3.2 Hardware Interfaces

The user will interact with the software using touch and keyboard.

3.3 Software Interfaces

- Django Rest Framework (Backend)
- React (Frontend)

- Flutter (Frontend)
- Tailwind CSS (Frontend)
- MySQL (Database)

3.4 Communications Interfaces

FTP will be used for file transferring, whereas for the chat function, messages will be end-to-end encrypted for security and privacy purposes.

4. System Features

1. Use case name

- User Registration
- Login / logout
- Add Exercise
- Add Tokens
- Check Analytics
- Add Workout
- Add Diet Plan
- Chat
- Track Progress
- Scan QR

2. Summary of this use case/ its usage

1- User Registration : GymAdmin can register users under GymUser & GymTrainer.

2- Login/Logout : MUST for the admin, user and trainer.

3- Add Exercise : GymAdmin can add new exercises for users and trainers to add to their workouts.

4- Add Tokens : GymAdmin can add tokens to a GymUser's account.

5- Check Analytics: GymAdmin can access business and other analytics.

6- Add Workout : This feature allows users to add an exercise that they've done to their profile including specifics (reps, time elapsed etc). This can be done by both GymUser and GymTrainer(to their client's profiles) in order to track progress.

7- Add Diet Plan : GymTrainer can design a diet plan for a client.

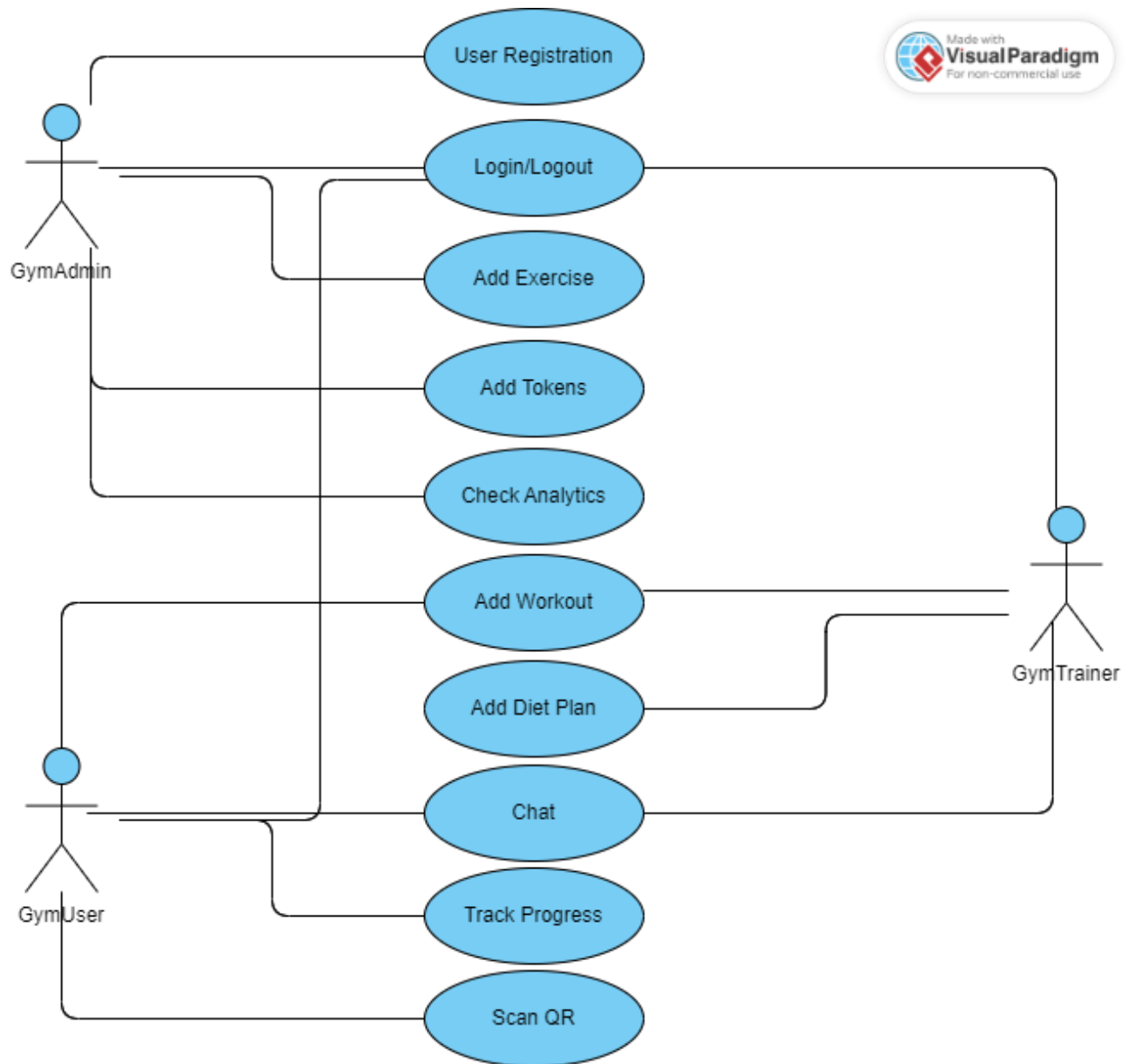
8- Chat : Users and trainers can chat amongst themselves.

9- Scan QR : GymUser have to use this when entering the gym to get a token deducted from their account.

3. Actors

- 1- GymAdmin
- 2- GymUser
- 3- GymTrainer

4. Use Case Diagram



4. Basic course of events/happy path

User Registration

ACTOR ACTION	SYSTEM RESPONSE
1- Admin will add user details	2- System will check if the all fields are valid
	3- System will display dashboard

Alternative path

If credentials are invalid then system will ask the user to re-enter their details

Post condition

Admin dashboard will be displayed with a pop up message “user has been registered”

Login

ACTOR ACTION	SYSTEM RESPONSE
1- User will enter credentials	2- System will check if the credentials are valid
	3- System will display dashboard

Alternative path

If credentials are invalid then system will ask the user to re-enter their details

Post condition

Home Page will be displayed.

Add Exercise

ACTOR ACTION	SYSTEM RESPONSE
1- Admin will add exercise details	2- System will check if the all compulsory fields are valid
	3- System will display dashboard

Alternative path

If any field is empty then system will ask the admin to fill that field.

Post condition

Admin dashboard will be displayed with a pop up message “Exercise added”

Add Tokens

ACTOR ACTION	SYSTEM RESPONSE
--------------	-----------------

1- Admin will choose user	3- System will update the user's tokens
2- Admin will enter the number of tokens	4- System will display choose user page

Alternative path

If there is a database connectivity error, system will display a try again message.

Post condition

'Choose user' page of the 'add tokens' page will be displayed.

Check Analytics

ACTOR ACTION	SYSTEM RESPONSE
1- Admin will press analytics button	2- System will display the analytics

Alternative path

System is unable to fetch analytics.

Post condition

Admin analytics page is displayed.

Add Workout

ACTOR ACTION	SYSTEM RESPONSE
1- User will search the exercise and add details	2- System will check if the all fields are valid
	3- System will display the workout added

Alternative path

If any field is empty then system will ask the user to fill that field.

Post condition

System will display the workout with the details added.

Add Diet Plan

ACTOR ACTION	SYSTEM RESPONSE
1- Trainer will select user	3- System will check if the all fields are valid
2- Trainer will add diet plan	3- System will display the diet plan

Alternative path

If any field is empty then system will ask the user to fill that field.

Post condition

System will display the plan with the details added.

Chat

ACTOR ACTION	SYSTEM RESPONSE
1- User will type message and hit send	2- System will relay the message to the recipient
	3- System will add message to chat

Alternative path

If there's a problem with the internet connection, message will not send.

Post condition

Message will be added to chat.

Scan QR

ACTOR ACTION	SYSTEM RESPONSE
1- User will scan the QR code while entering the gym	2- A token will be deducted from their account
	3- System will display dashboard

Alternative path

If the camera is shaky, an error will be shown

Post condition

A token will be deducted from the user's account.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- This application will be reliable and responsive.
- Cross-device efficiency will be prioritized.
- Performance on older devices will be optimized.
- HTTP and TCP will be implemented for optimal network connectivity.

5.2 Safety Requirements

There are no such safety requirements that hence be observed in utilization of this project.

5.3 Security Requirements

Messages will be end-to-end encrypted. User authentication module with proper validations has been added to this application.

5.4 Software Quality Attributes

Robustness: It should be robust in terms that it could handle a lot of users simultaneously without having any performance issues.

Usability: It should be easy to use for a normal citizen without having any heavy technical knowledge.

Reliability: we will use try catch to handle exceptions to secure the application from crashing.

Reusability: classes and functions will be reused in implementations.

5.5 Business Rules

Tokens if not used for more than 3 months will expire.

6. Other Requirements

Appendix A: Glossary

Django

Django is a high-level web framework written in Python that encourages rapid development and clean, pragmatic design. It follows the Model-View-Controller (MVC) architectural pattern, but in Django, it's often referred to as the Model-View-Template (MVT) pattern.

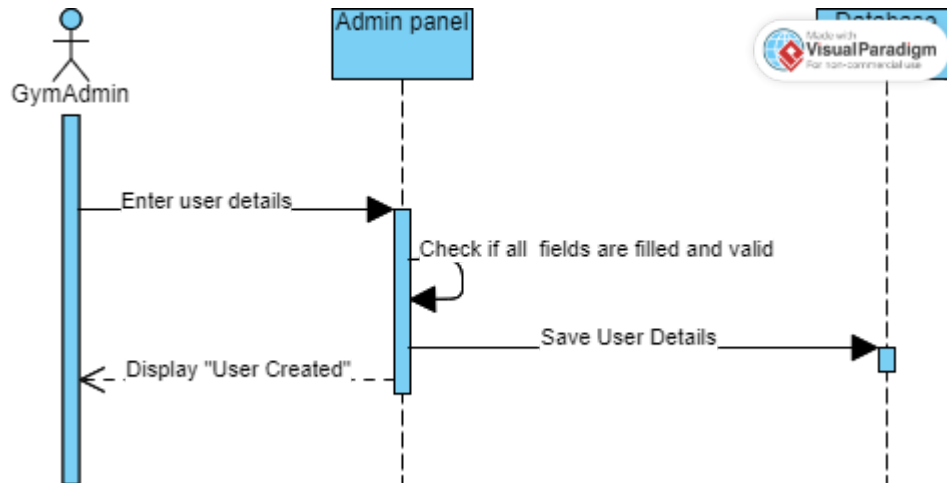
Django Rest Framework

Django Rest Framework (DRF) is a powerful and flexible toolkit for building Web APIs in Django web applications. It is an extension to Django that provides additional features specifically for designing, building, and testing RESTful APIs. DRF is not a separate framework; rather, it is built on top of Django and extends its capabilities.

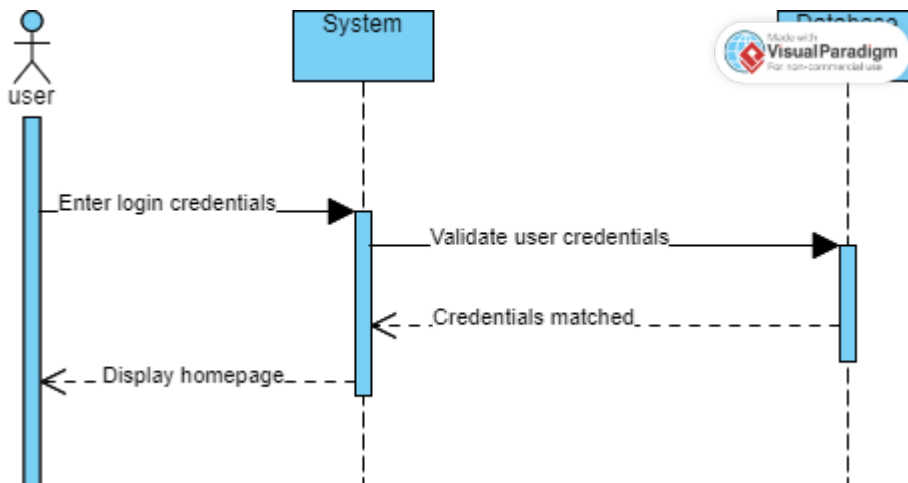
Appendix B: Analysis Models

6.1 System Sequence Diagrams

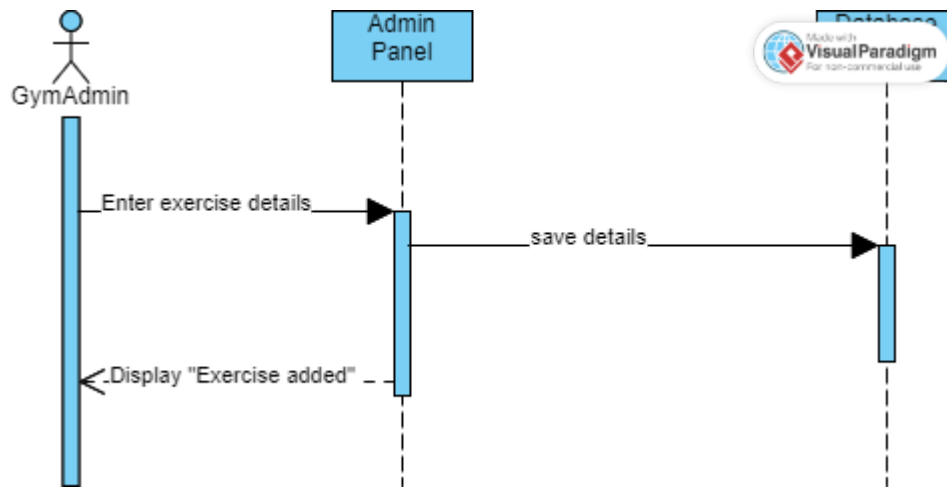
User Registration:



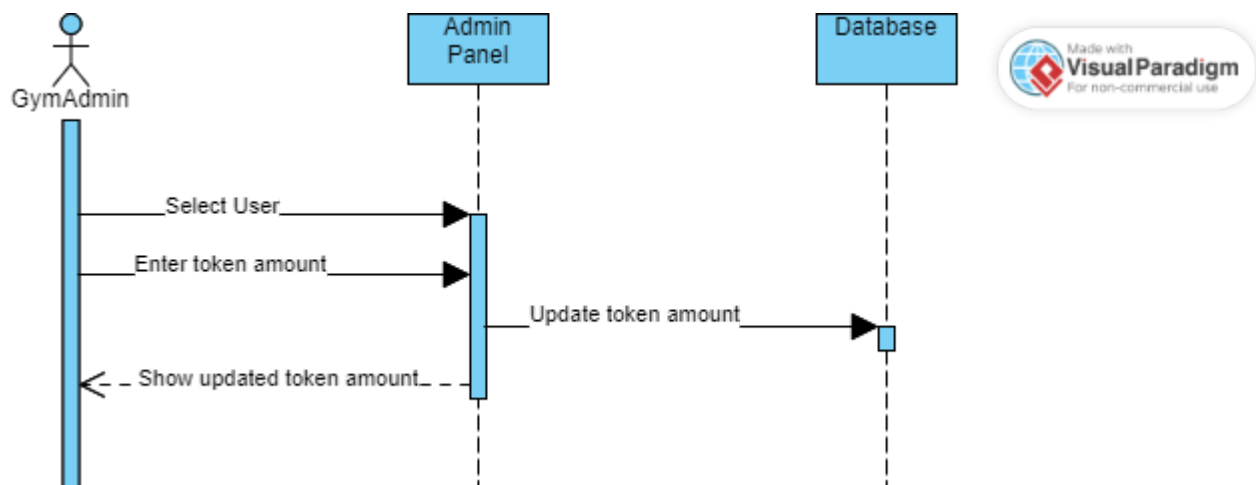
Login:



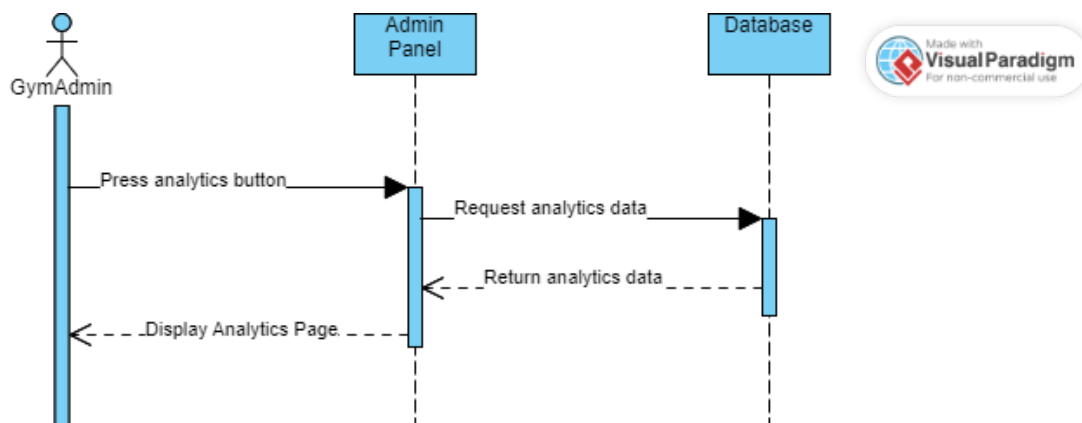
Add Exercise:



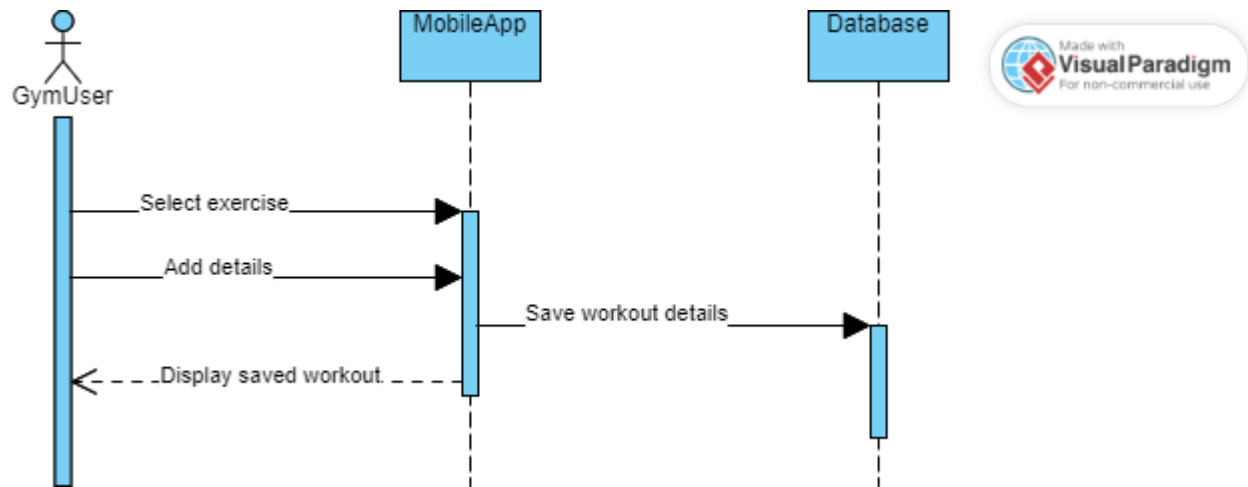
Add Token:



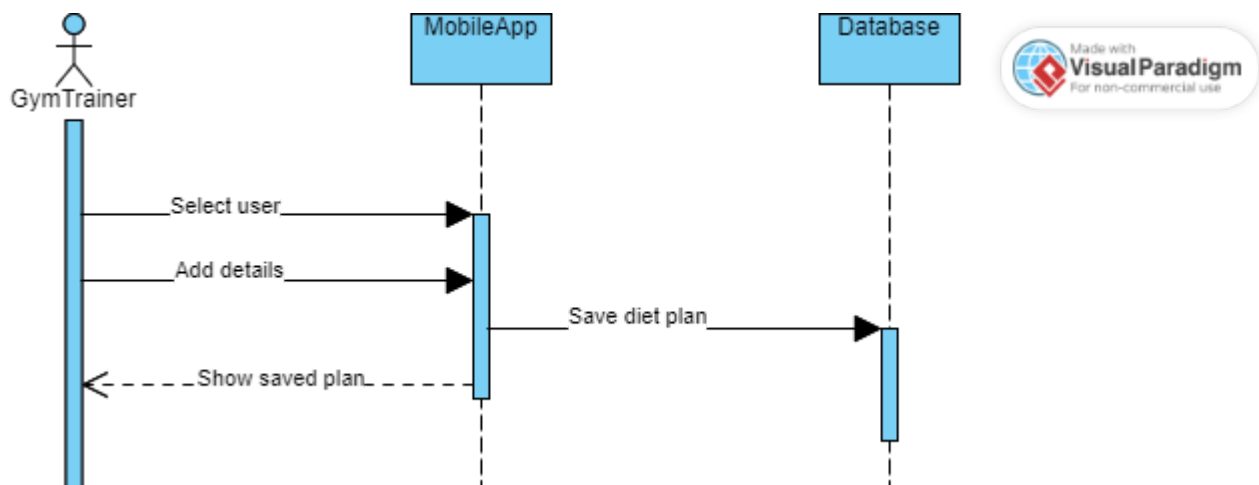
Check Analytics:



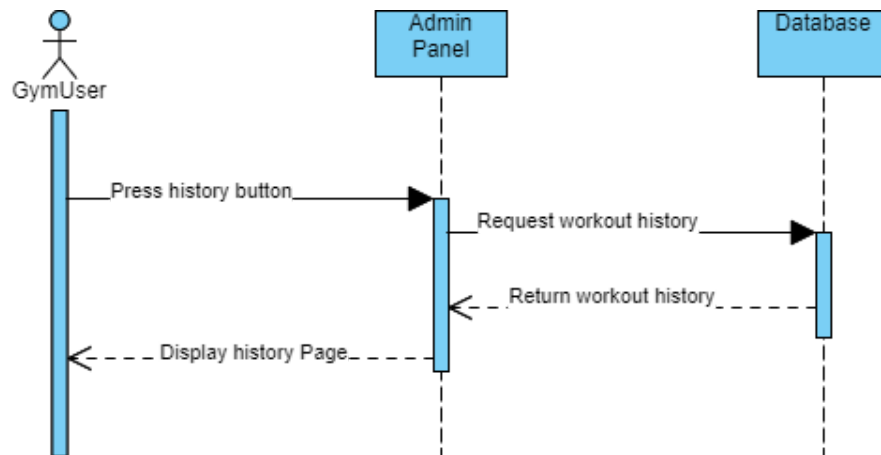
Add Workout:



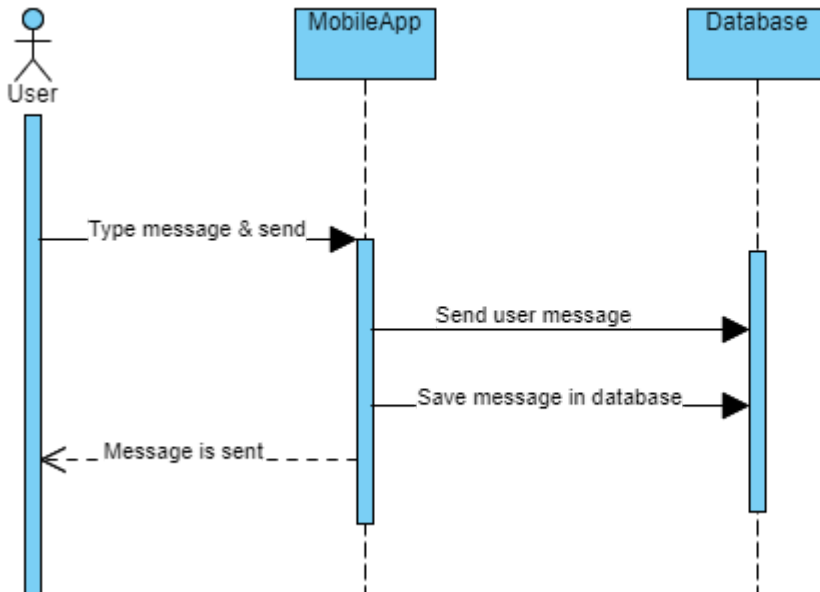
Add Diet Plan:



Track Progress:



Chat:



Software Design Specification

1. Introduction

1.1 Purpose of this document

The SDS focuses on specifying a high-level view of the architecture of our system, and on the interaction between the user and the system. It also focuses on detailing a low-level view of each component of the software and how the components interact with each other.

The objective of this project is development of a unified framework for gyms that can help the gym owners, trainers and members likewise. NexFit works by providing the trainers and members with a mobile application that can be used to communicate among them, track progress, etc. The members have tokens that are deducted from their account every time they work out at the gym. The gym admins can track all this through a web application.

1.2 Scope of the development project

NexFit stands as a cutting-edge fitness management solution revolutionizing the fitness industry. Its core design features three distinct user roles: Superusers, GymAdmins, GymTrainers, and GymUsers. Superusers, equipped with the power of the Django admin panel, oversee the entire platform. GymAdmins, selected by Superusers, manage GymUsers efficiently through a dedicated React admin interface. GymUsers, the end clients, experience a personalized fitness journey via a feature-rich mobile application. The platform introduces a token system, ensuring a reliable attendance system that helps the user to save money. Beyond basic functionalities, NexFit offers tailored fitness plans, exercise routines, diet management, and real-time chat functionality for direct communication between trainers and users.

1.3 Definitions, acronyms, and abbreviations

Not Applicable.

1.4 References

Not Applicable.

1.5 Overview of document

The next sections will describe the architectural design of the project, the high level components and their interactions. Some issues will be discussed which might be faced in the user interface. The system architecture description section describes the overview of the components, structure and relationships among the components along the way it also identifies the user interface issues.

On the other hand, section 3 gives in depth description of components, section 4 gives overall description of the interface of the system, section 5 identifies the relations with other products, section 6 describes the design trade offs.

2. System architecture description

This section provides an overview and rationale for the program's data and architectural design decisions.

2.1. Section Overview

General constraints that are applicable to the system and have impact on the system are discussed in the section hereunder. The data design and structure design of the system are also part of this section of the document.

2.2. General Constraints

Technologies:

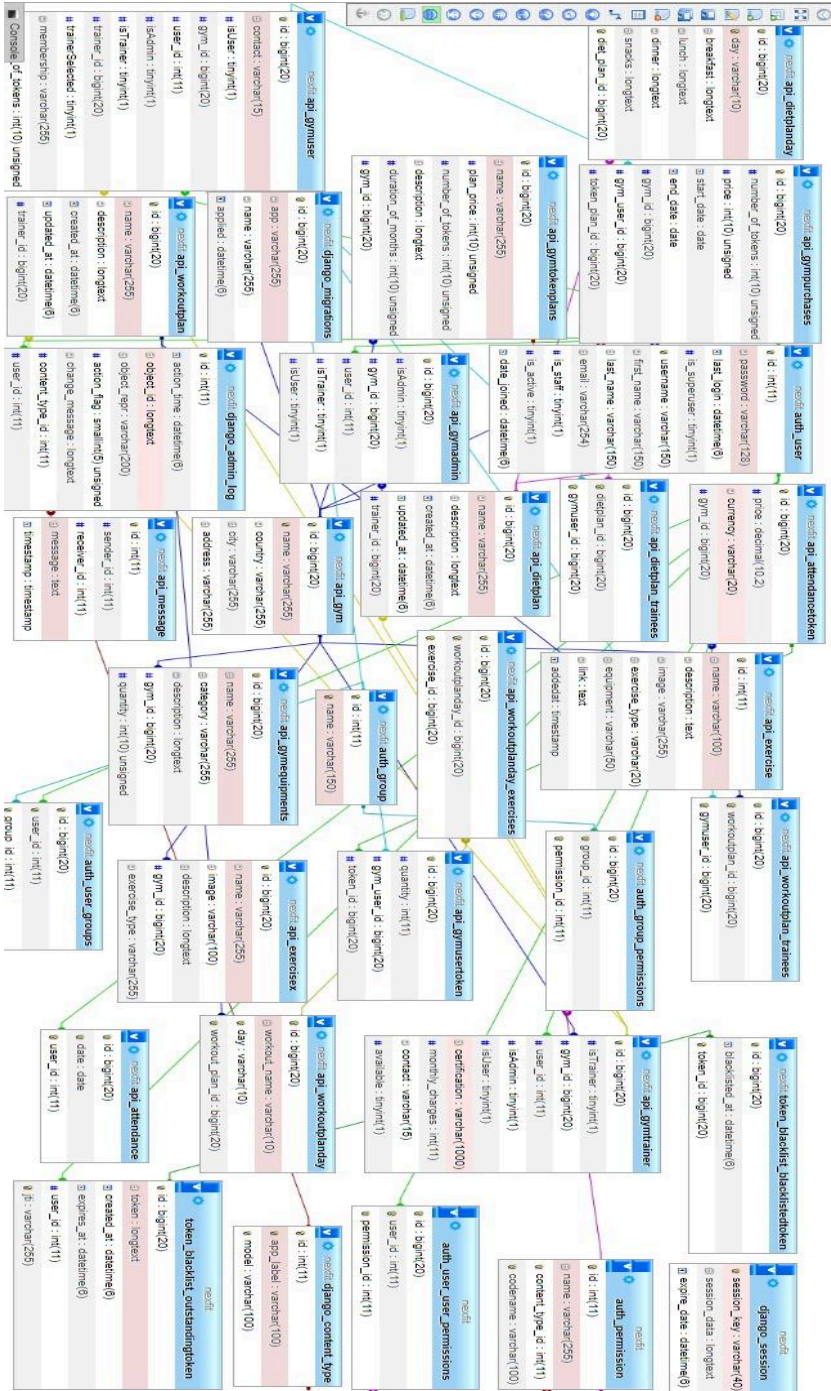
- a. Django Rest Framework (Backend)
- b. React (Frontend)
- c. Flutter (Frontend)
- d. Tailwind CSS (Frontend)
- e. MySQL (Database)

Tools:

- a. Visual Studio Code
- b. MySQL
- c. Android Studio

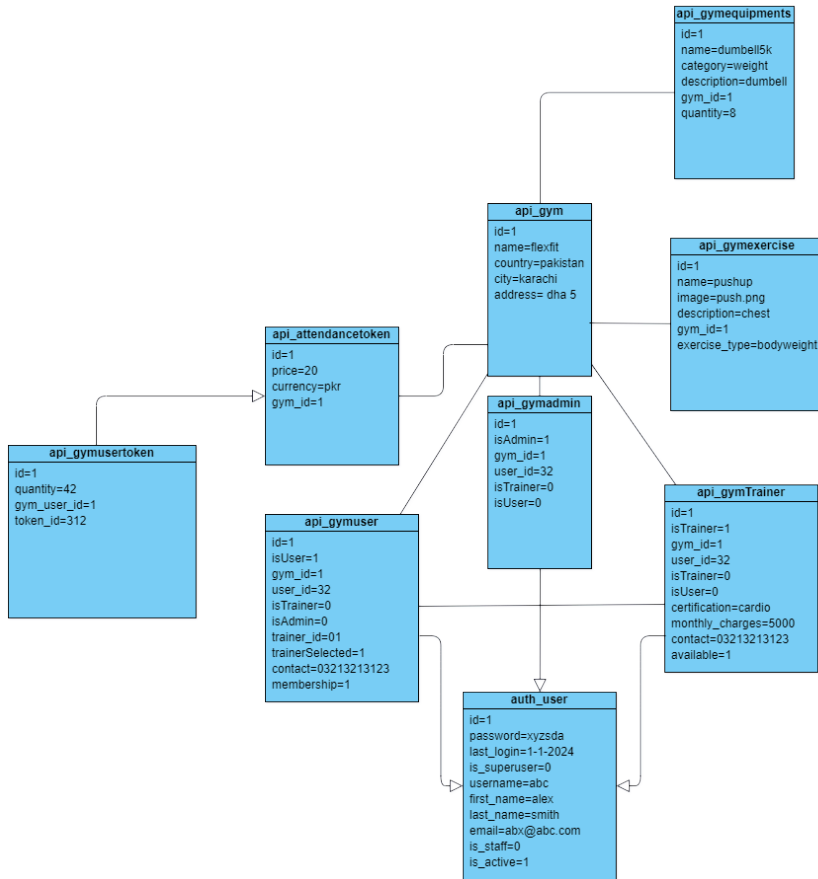
2.3. Data Design

Database offers an efficient and uniform method of storing and accessing the user information along with information regarding the projects stored, their development language and publishing privacy.



For the product NexFit, the database that is used is MySQL. For an overview of the database, an Entity Relation Diagram has been attached below

2.4. Program Structure



3. Detailed description of components

3.1. Section Overview

In this section we describe the details of components in the system, it will also give the overall view of all the functional components. It will describe each component's identification, type, purpose, function subordinates, Dependencies, interfaces, resources, processing and data.

3.2. Component n Detail

3.2.1. User Registration

Identification	User Registration
Type	Class/Page/Method
Purpose	User registration into the system
Function	This screen allows GymAdmin to enter all the details needed to register a new user.
Subordinates	-
Dependencies	The user registration screen depends on entered details which ties to another component i.e. the creation of an account and allowing the registered user to access the mobile application
Interfaces	The screen is designed to give credentials of the users to the system which provides the services to the users for further priorities and procedure
Resources	Software requirements: SQL Server, Visual studio code. Operating system: Windows 7 or above. Database Requirements: It validates the users that exist in the database.
Processing	The data entered is saved to the database.
Data	Data required for the Signup Screen is the name, email, phone number, password, trainer and membership status.

3.2.2. Login

Identification	Login
Type	Class/Page/Method
Purpose	User authentication into the system
Function	This screen allows the user to enter user credentials and verifies, if verified the user is taken to its homepage.
Subordinates	-
Dependencies	The login screen depends on entered email and password which tie to another component that is the dashboard that enables use of gym services.
Interfaces	The screen is designed to give credentials of the users to the system which provides the services to the users for further priorities and procedure.
Resources	Software requirements: SQL Server, Visual studio code. Operating system: Windows 7 or above. Database Requirements: It validates the users that exist in the database.
Processing	This screen requires email(username) and password for verifying the account and allowing access to NexFit.
Data	Data required for the Login Screen is the email and password. The database entries of users have access to the application.

3.2.3. Add Exercise

Identification	Add exercise
----------------	--------------

Type	Class/Page/Method
Purpose	Add new exercises for members and trainers to add to their workouts..
Function	This screen allows GymAdmin to add exercise details needed for a new exercise.
Subordinates	Add exercise details
Dependencies	-
Interfaces	Exercise details form
Resources	Software requirements: SQL Server, Visual studio code. Operating system: Windows 7 or above. Database Requirements: It validates the users that exist in the database.
Processing	The data entered is saved to the database.
Data	Data required for the Signup Screen is the exercise details.

3.2.4. Add Tokens

Identification	Add tokens
Type	Class/Page/Method
Purpose	Add tokens to user accounts.
Function	This screen allows GymAdmin to add tokens to a user's account.
Subordinates	- Select user - Add number of tokens
Dependencies	The selected user should have an active membership.
Interfaces	Add token screen
Resources	Software requirements: SQL Server, Visual studio code. Operating system: Windows 7 or above. Database Requirements: It validates that the user exists in the database.
Processing	The number of tokens are added to the amount of tokens in the user's account in the database.
Data	Data required is the number of tokens.

3.2.5. Check Analytics

Identification	Check analytics
Type	Class/Page/Method
Purpose	Check gym analytics
Function	This screen allows the GymAdmin to review the gym analytics.
Subordinates	-
Dependencies	The gym should be active.
Interfaces	Analytics page
Resources	Software requirements: SQL Server, Visual studio code. Operating system: Windows 7 or above. Database Requirements: It gets the numbers from the database.
Processing	The analytics are calculated and displayed.
Data	Data is fetched from the database.

3.2.6. Add Workout

Identification	Add workout
----------------	-------------

Type	Class/Page/Method
Purpose	Add new workout progress.
Function	This screen allows the user or trainer to add a workout to the user's progress.
Subordinates	- Select exercise. - Add details. - Add another exercise
Dependencies	The user should be logged into the application.
Interfaces	Add workout page.
Resources	Software requirements: SQL Server, Visual studio code. Operating system: Android 10 or above, iOS 15 or above. Database Requirements: It validates that the user exists in the database.
Processing	Add the workout to the database.
Data	Data required is the exercise(s) id, no. of sets and no. of reps per set.

3.2.7. Add Diet Plan

Identification	Add diet plan
Type	Class/Page/Method
Purpose	Add a diet plan
Function	This screen allows the trainer to add diet plan to a user's profile.
Subordinates	- Select user - Add diet plan
Dependencies	The trainer should be logged into the application.
Interfaces	Add diet plan page.
Resources	Software requirements: SQL Server, Visual studio code. Operating system: Android 10 or above, iOS 15 or above. Database Requirements: It validates that the trainer exists in the database.
Processing	Add diet plan to the selected user's account in the database.
Data	Data required is the diet details.

3.2.8. Chat

Identification	Chat
Type	Class/Page/Method
Purpose	Chat between user and trainer.
Function	This screen allows the user or the trainer with each other.
Subordinates	-
Dependencies	The user/trainer should be logged into the application.
Interfaces	Chat screen.
Resources	Software requirements: SQL Server, Visual studio code. Operating system: Android 10 or above, iOS 15 or above. Database Requirements: It validates that the trainer exists in the database.
Processing	The message is encrypted, sent to the receiver and decrypted. The chat is updated on both ends.
Data	Data require is the message.

3.2.9. Scan QR

Identification	Scan QR
Type	Class/Page/Method
Purpose	Scan QR code to deduct token.
Function	This screen opens the camera for the user to scan QR code and deduct a token from their account.
Subordinates	-
Dependencies	The user should be logged into the application. Camera access should be allowed.
Interfaces	Camera with a overlay to properly frame QR code.
Resources	Software requirements: SQL Server, Visual studio code. Operating system: Android 10 or above, iOS 15 or above. Database Requirements: It validates that the user exists in the database.
Processing	Scan QR and deduct token from database.
Data	-

4. User Interface Design

4.1. Section Overview

This section provides insight to the user interface of the product NexFit. The detailed description of the user interface components and why they were chosen for this product. The idea behind this user interface and its components is briefly discussed in this section.

4.2. Interface Design Rules

The design rules used in our interface are as follows:

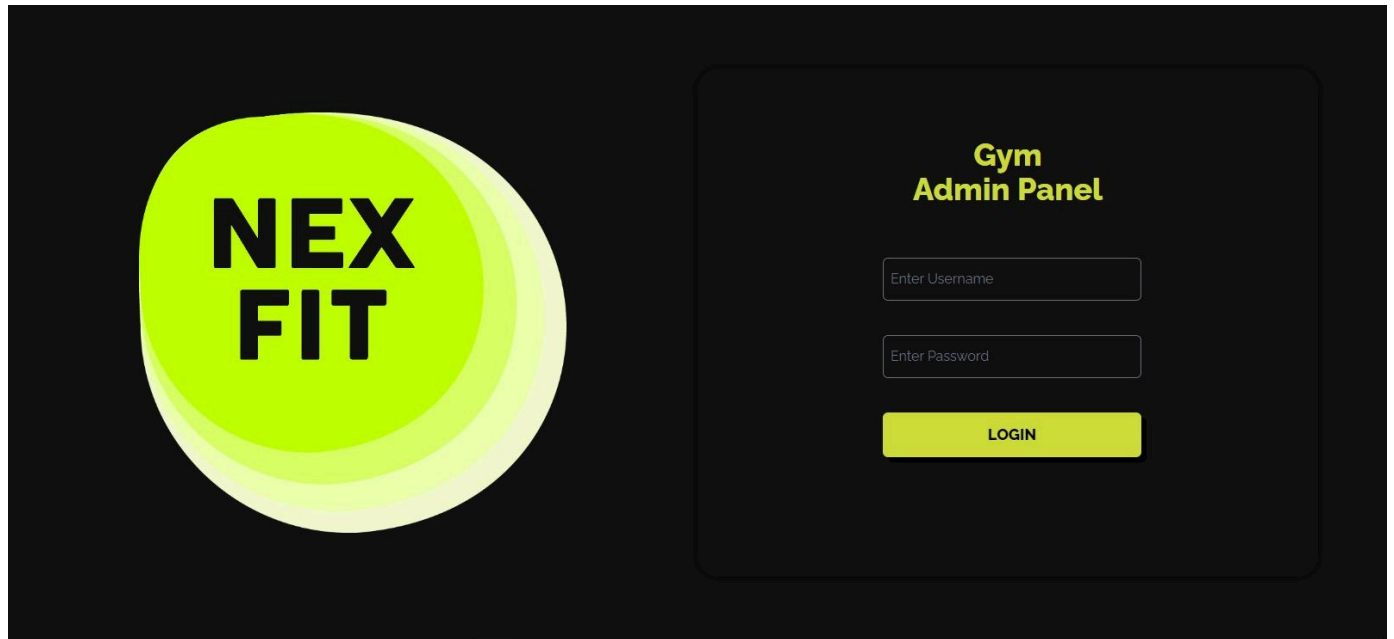
- The interface is flexible i.e. it allows the user to either use keyboard and or mouse.
- Contains buttons that are easy to use.
- Labels are of reasonable size.
- The interface is user friendly, it allows convenient usability even for non-traditional users.
- Displays descriptive texts and messages where necessary to help users identify any requirements.
- It provides visual cues.

4.3. GUI Components

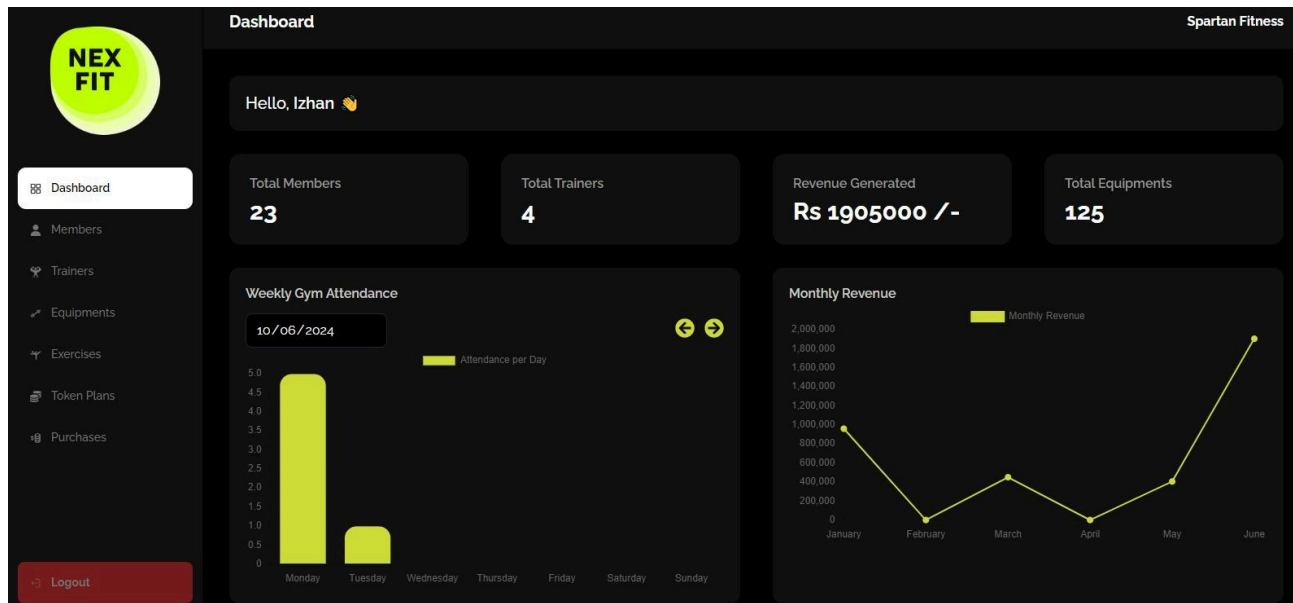
One of the main components of this product is the django rest framework.

4.4. Detailed Description

4.4.1. Admin Login



4.4.2. Dashboard



4.4.3. Add Member



- Dashboard
- Members
- Trainers**
- Equipments
- Exercises
- Token Plans
- Purchases

Logout

Add User

Spartan Fitness

NEW MEMBER

Enter Username

Enter Email

Enter First name

Enter Last Name

Enter Contact Number

Enter Password

No Trainer

Create

4.4.4. Add Trainer



- Dashboard
- Members
- Trainers**
- Equipments
- Exercises
- Token Plans
- Purchases

Logout

New Trainer

Spartan Fitness

Enter Username

Enter Email

Enter First name

Enter Last Name

Enter Contact Number

Enter Password

Enter Certification

Enter Monthly Charges

Create

4.4.5. Members List



- Dashboard
- Members**
- Trainers
- Equipments
- Exercises
- Token Plans
- Purchases

Spartan Fitness

+ New Member

ID	Username	Name	Email	Contact	Trainer	Tokens	Edit
3	user09	user 09	user09@gmail.com	0909090909	09	409	
4	user10	user 10	user10@gmail.com	1010101010	10	75	
5	user11	user 11	user11@gmail.com	1111111111	11	325	
6	user12	user 12	user12@gmail.com	1212121212	12	0	
9	user14	user 14	user14@gmail.com	1414141414	14	0	
10	user15	user 15	user15@gmail.com	1515151515	15	0	
11	user16	user 16	user16@gmail.com	1616161616	16	0	
12	user18	user 18	user18@gmail.com	1818181818	18	0	
13	user19	user 19	user19@gmail.com	1234567890	19	0	

4.4.6. Trainers List



- Dashboard
- Members
- Trainers**
- Equipments
- Exercises
- Token Plans
- Purchases

Spartan Fitness

+ New Trainer

ID	Username	Name	Email	Contact	Certification	Monthly Charges
1	trainer02	Sunny Deol	trainer02@gmail.com	222222222	Fitness Instructor	PKR 15000 /-
2	trainer03	Bobby Deol	trainer03@gmail.com	333333333	Certification	PKR 5000 /-
3	trainer04	Trainer 04	trainer04@gmail.com	0404040404	None	PKR 6900 /-
4	muscleman	Body Builder	muscleman@nexusfit.com	6969696969	Certified MuscleMan	PKR 69000 /-

Logout

4.4.7. Add Equipment



- Dashboard
- Members
- Trainers
- Equipments**
- Exercises
- Token Plans
- Purchases

Logout

Add Equipment

Spartan Fitness

Enter Equipment Name

Enter Equipment Category

Enter Equipment Quantity

+

-

Enter Equipment Description

ADD

4.4.8. Equipment List



- Dashboard
- Members
- Trainers
- Equipments**
- Exercises
- Token Plans
- Purchases

Logout

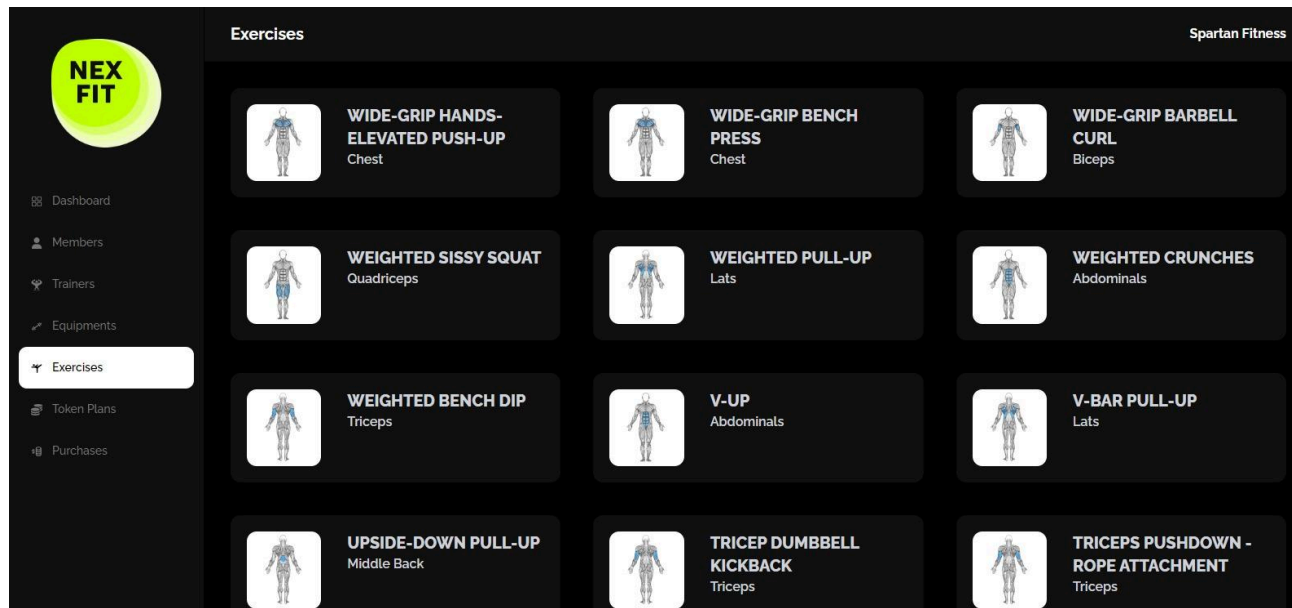
Equipments

Spartan Fitness

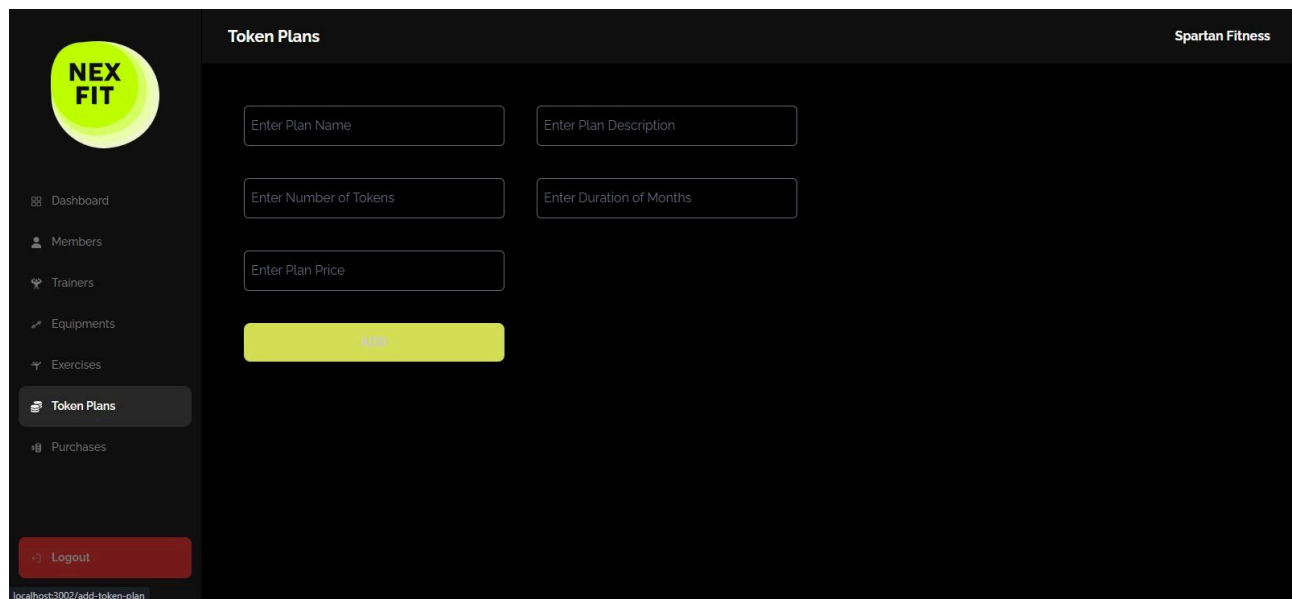
+ New Equipment

ID	Name	Category	Description	Quantity
1	Dumbbells	Weight Lifting	Hand and free weights	25
2	Barbell	Weight lifting	Used for various purposes	100

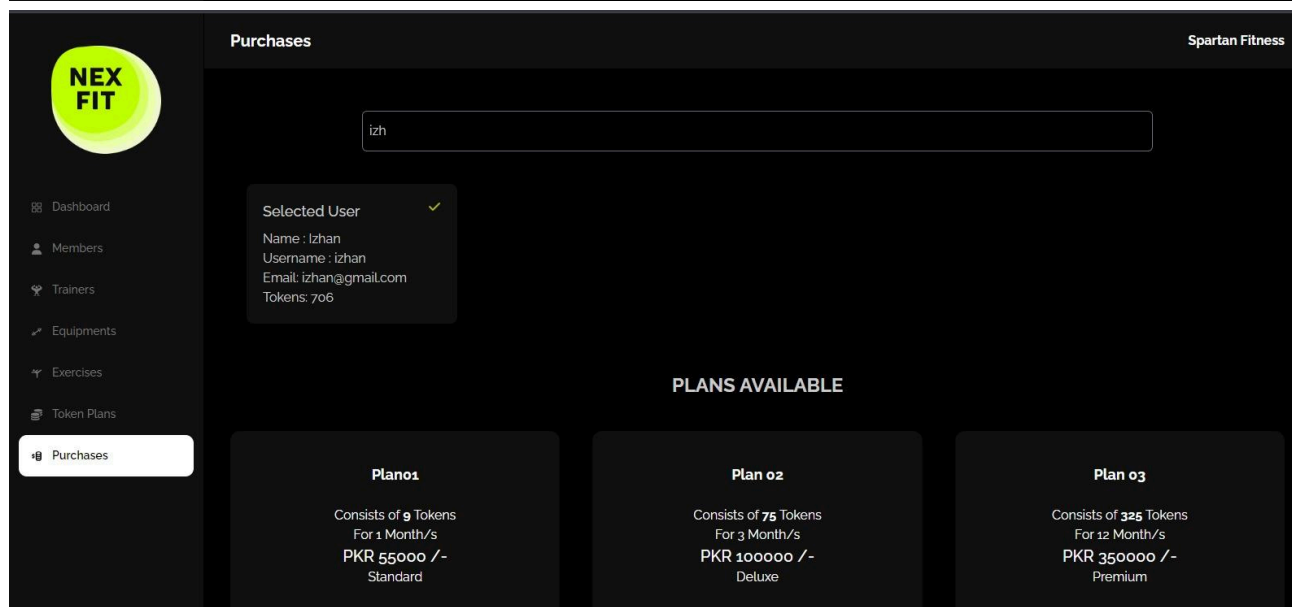
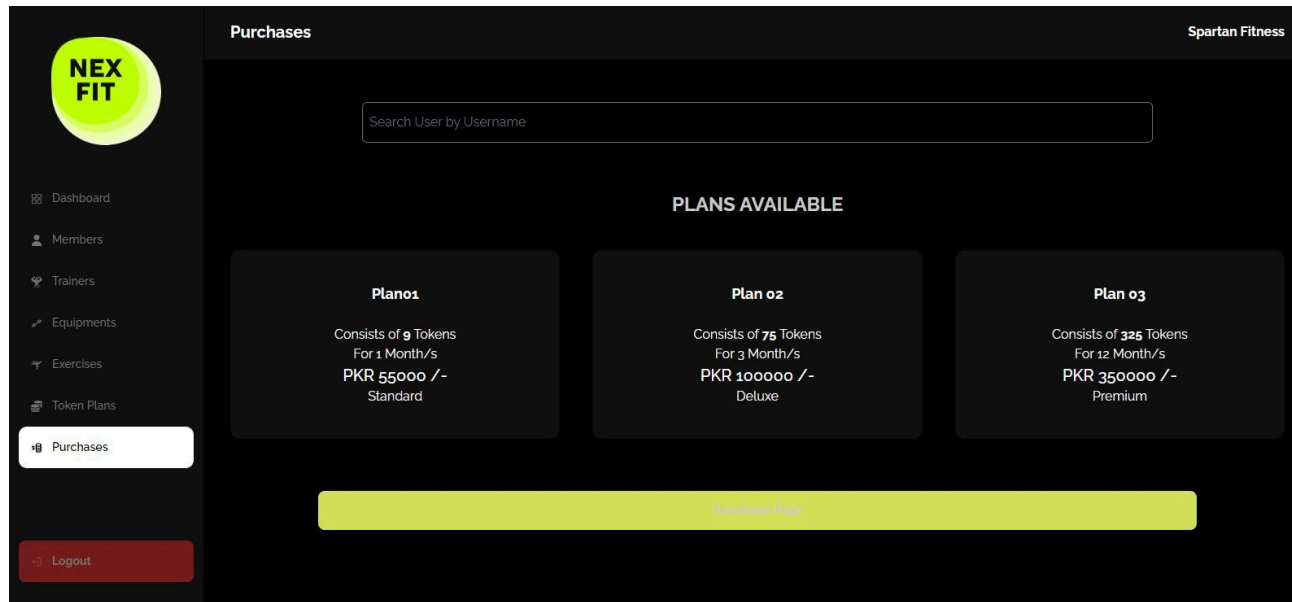
4.4.9. Exercises List



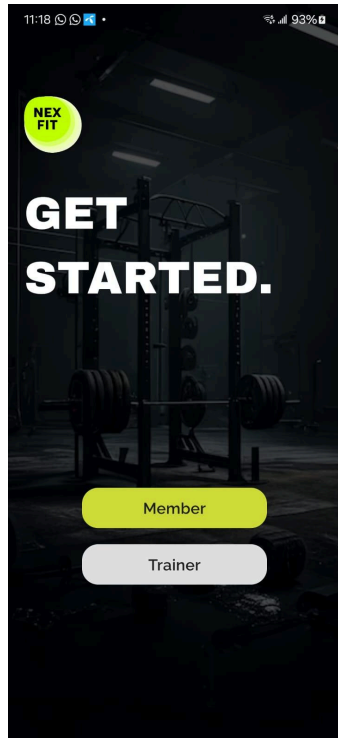
4.4.10. Add Token Plan



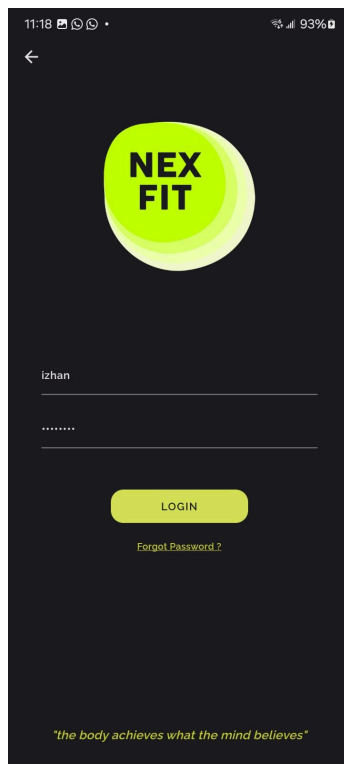
4.4.11. Purchase Tokens



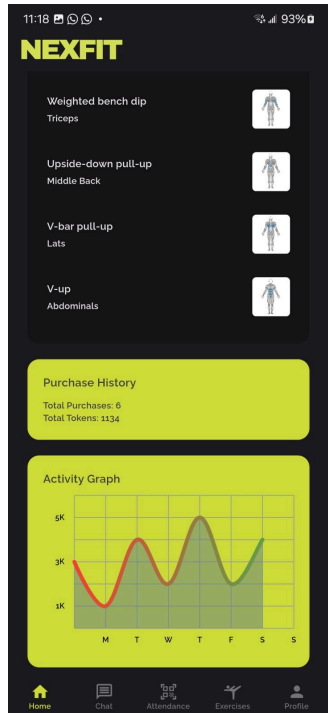
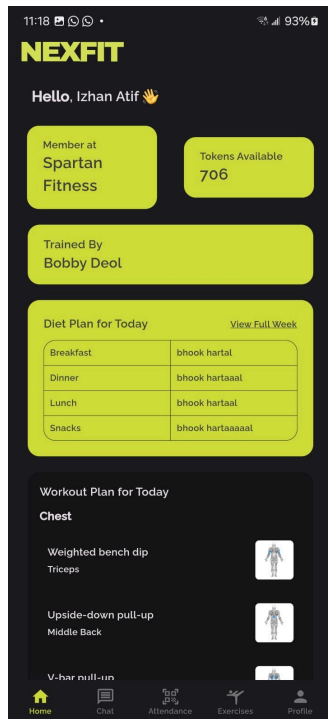
4.4.12. Mobile Application Home



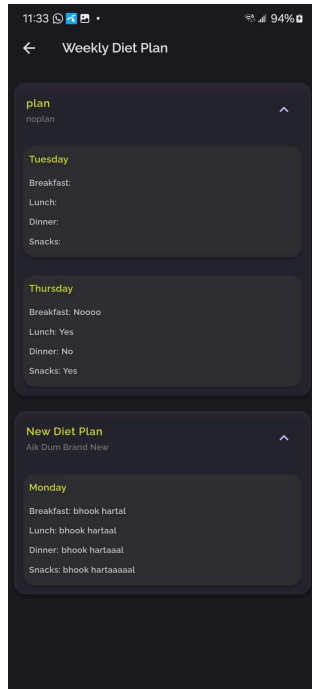
4.4.13. Login



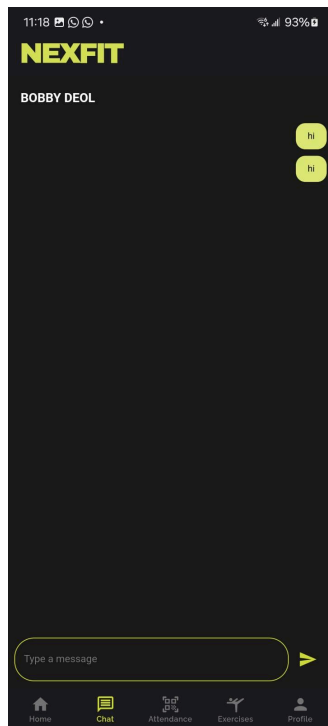
4.4.14. Member Home



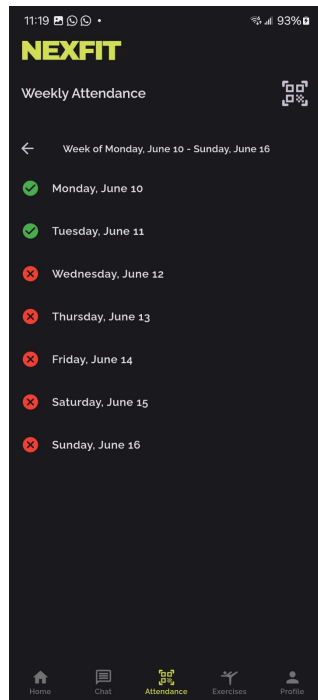
4.4.15. Member Diet Plan



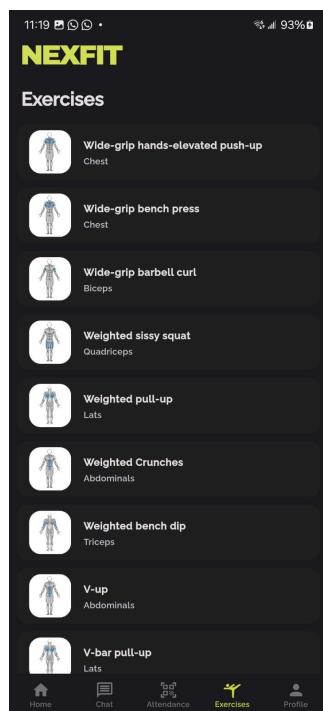
4.4.16. Member Chat



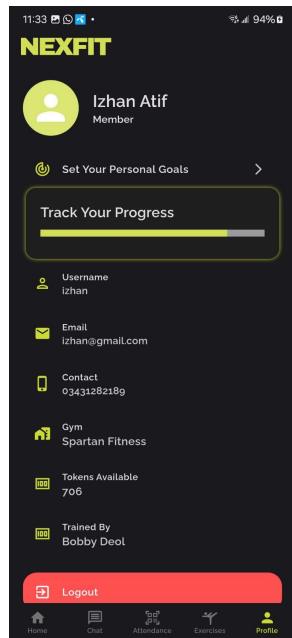
4.4.17. Member Attendance



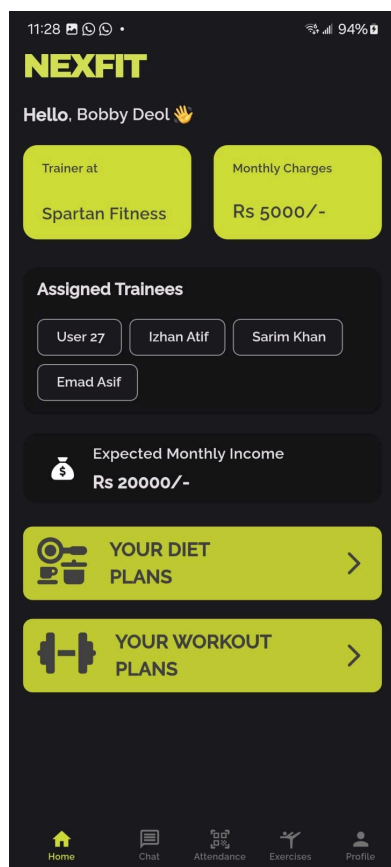
4.4.18. Exercises



4.4.19. Member Profile



4.4.20. Trainer Home



4.4.21. Workout Plan



11:40

94%

←
Edit Workout Plan

Name
Beginner

Description
Workout Plan for Beginners

Assigned Trainees:

Unknown

Izhan

Add Trainee

Workout Days:

Monday

Workout Name
Chest

Weighted bench dip

–

Upside-down pull-up

–

V-bar pull-up

–

V-up

–

Add Exercise

Add Another Day

Update Workout Plan

11:29

93%

←
Add Workout Plan

Plan Name

Description

Assigned Trainees:

Add Trainee

Workout Days

Day
day_1

Workout Name

Exercises

Weighted pull-up

🗑️

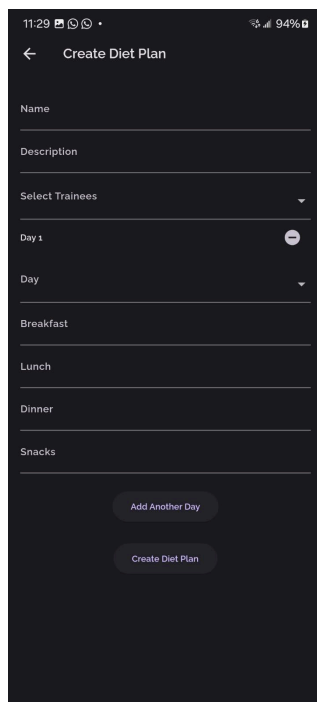
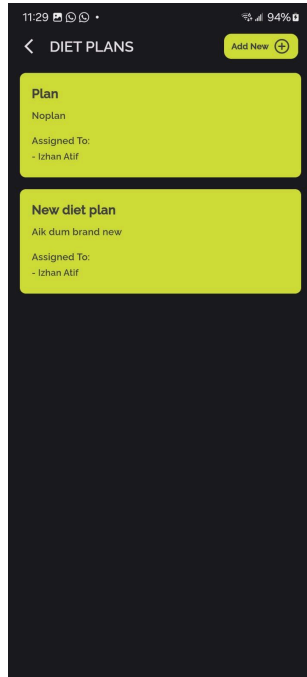
Add Exercise

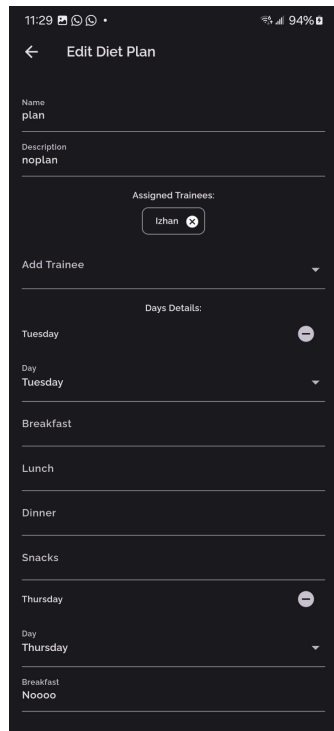
Remove Day

Add Workout Day

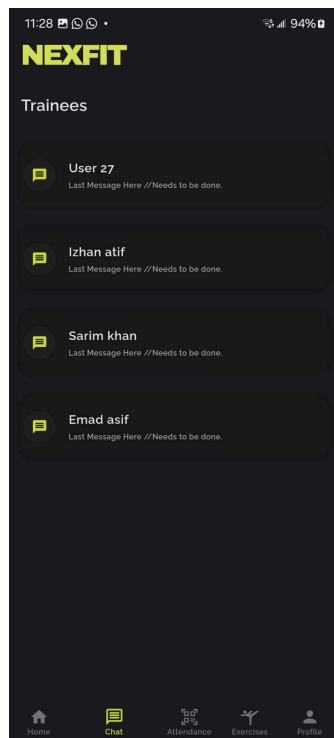
Create Workout Plan

4.4.22. Diet Plan

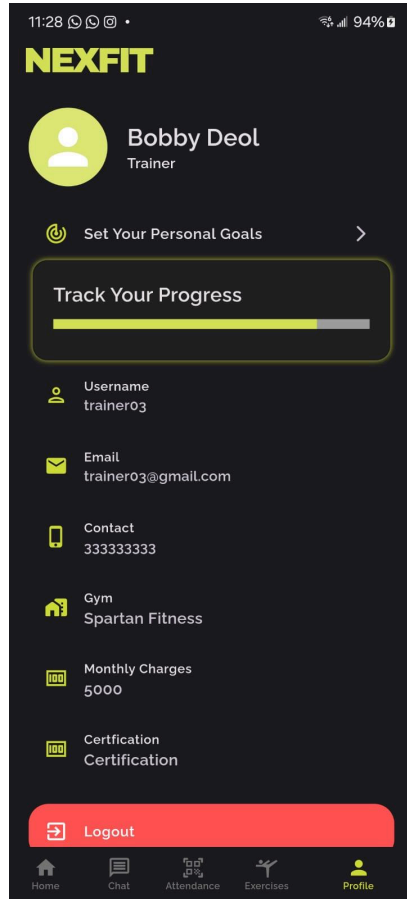




4.4.23. Trainer Chat



4.4.24. Trainer Profile



5. Reuse and relationships to other products

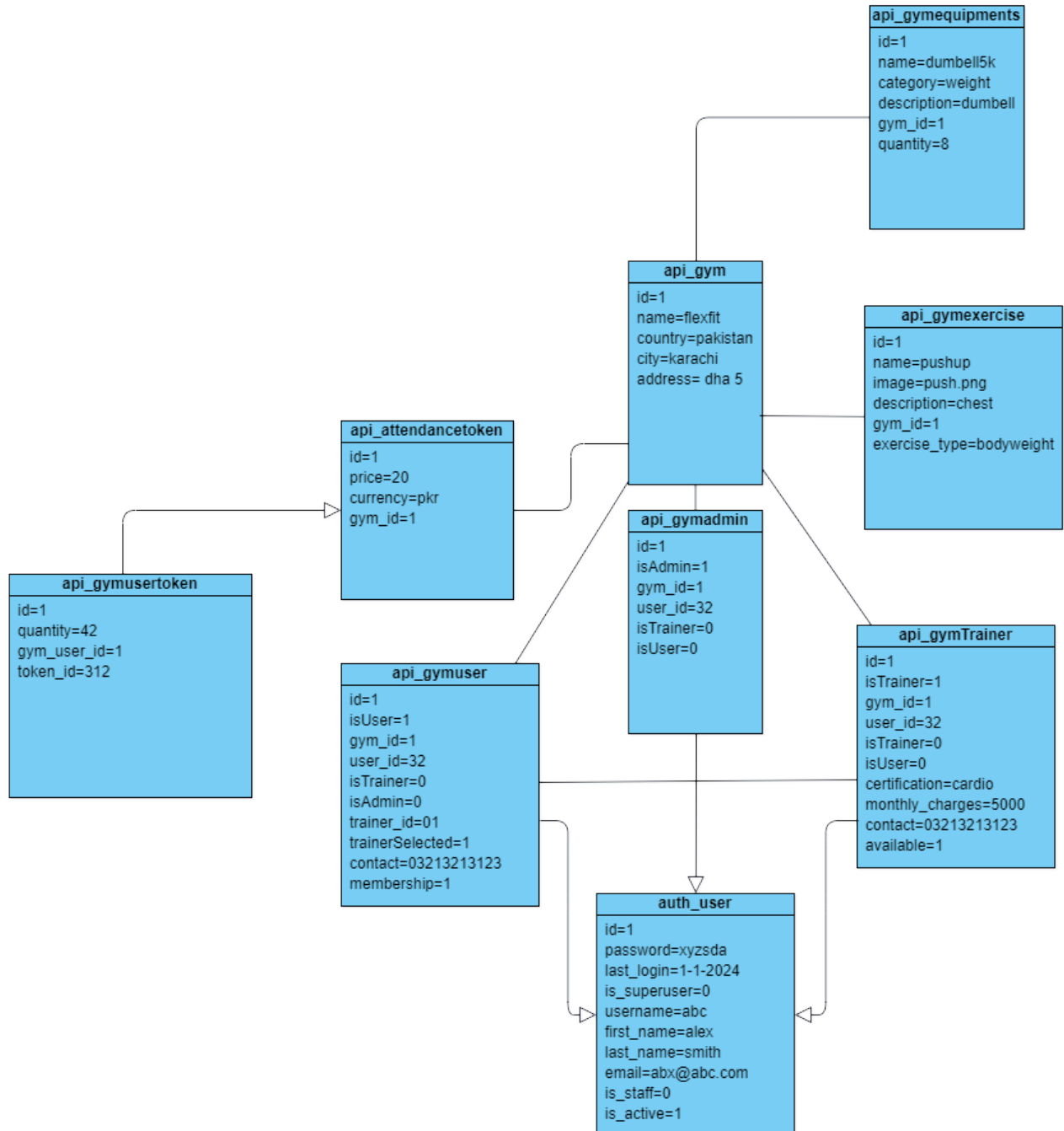
The application has been developed using visual code studio which allows easy changes, implementation and deployment. Our app remained integrated with all the other components that 59 allows reuse of functions, database connection, database and classes etc. reducing overall redundancy.

6. Design decisions and tradeoffs

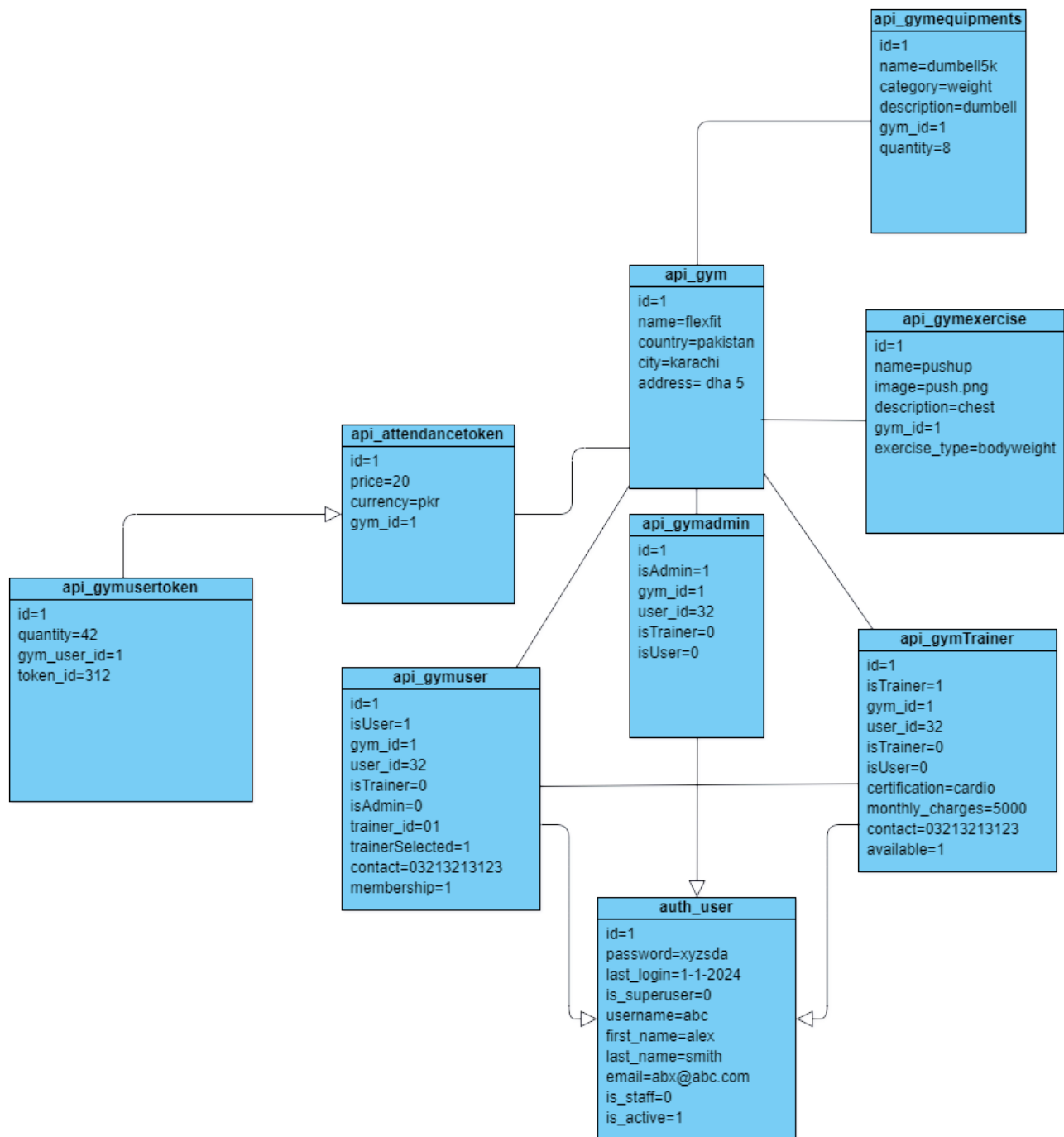
Object oriented software development approach has been adopted for the development of the software. Model view controller architecture has been used to make the communication robust and efficient. The communication takes place through the HTTP protocol which means that an internet connection is required.

7. Appendices

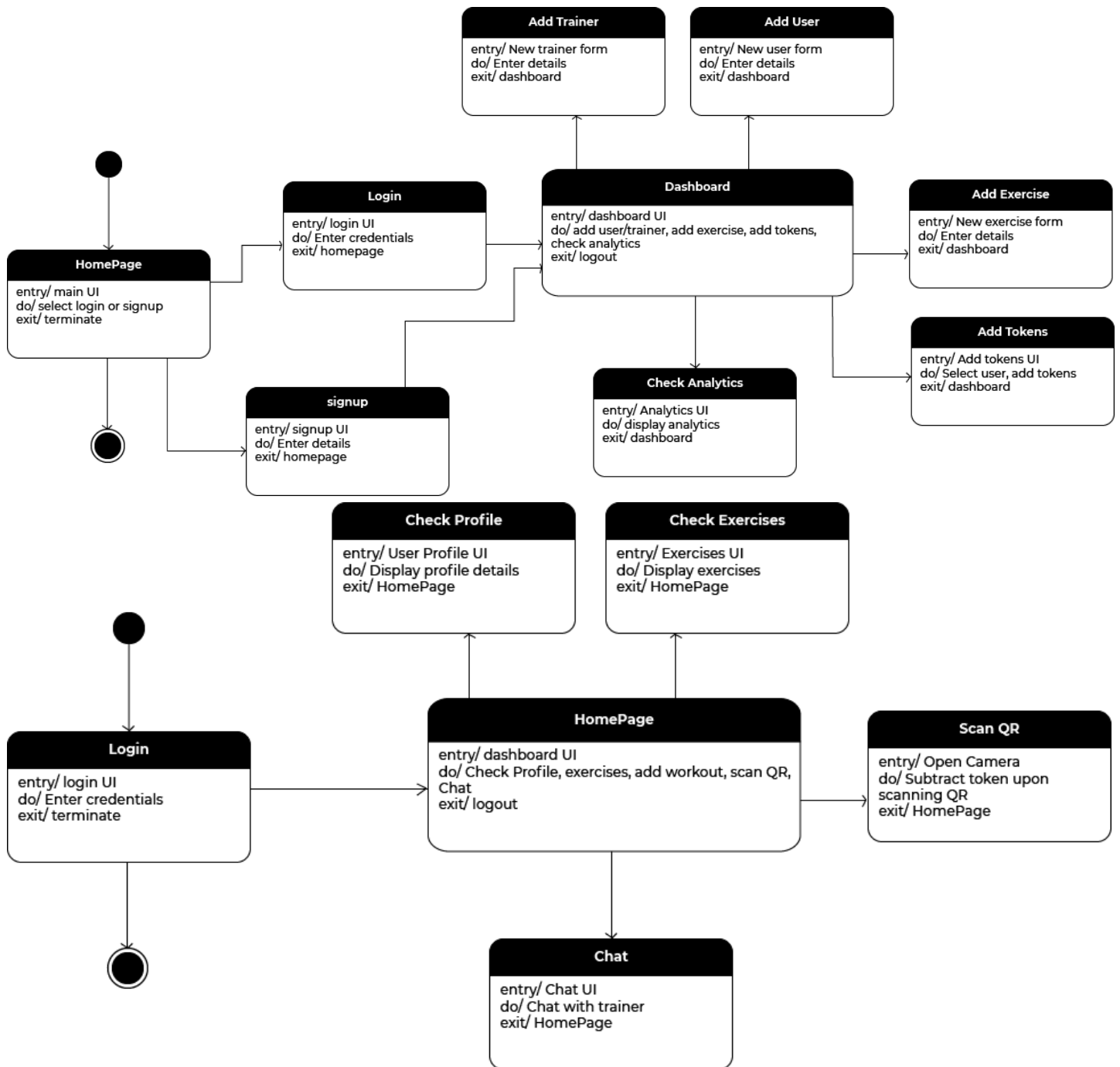
7.1. Class Diagram



7.2. Object Diagram

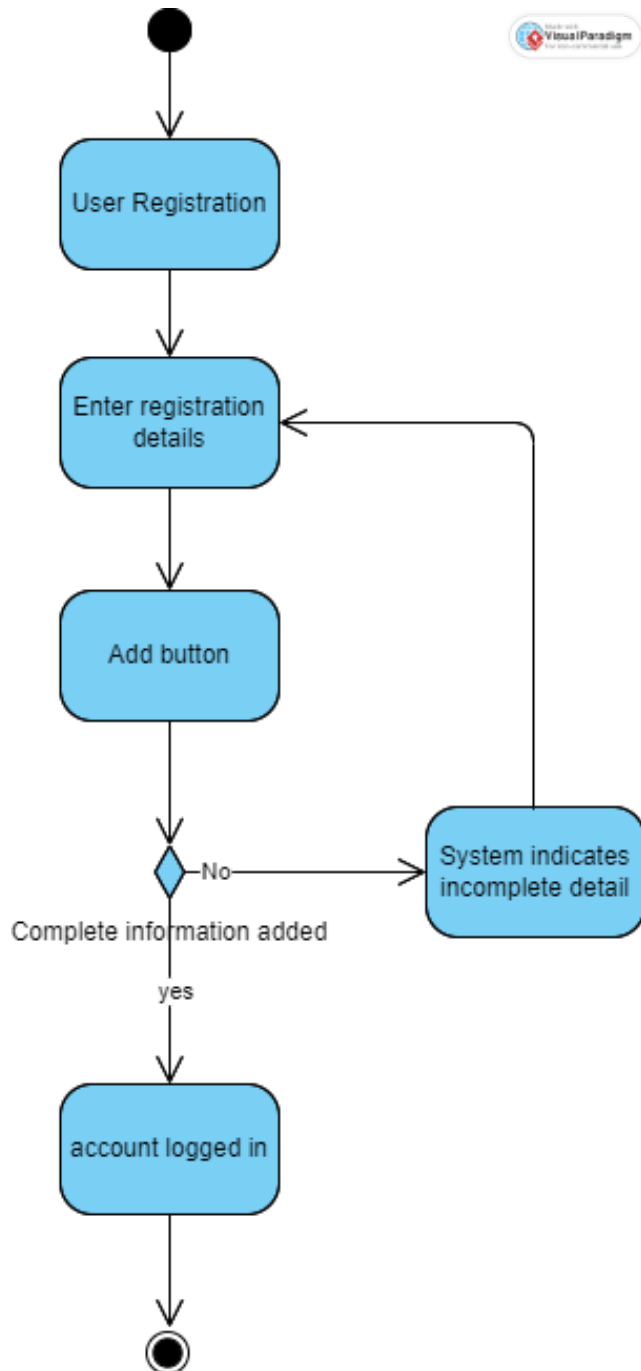


7.3. State Chart Diagram

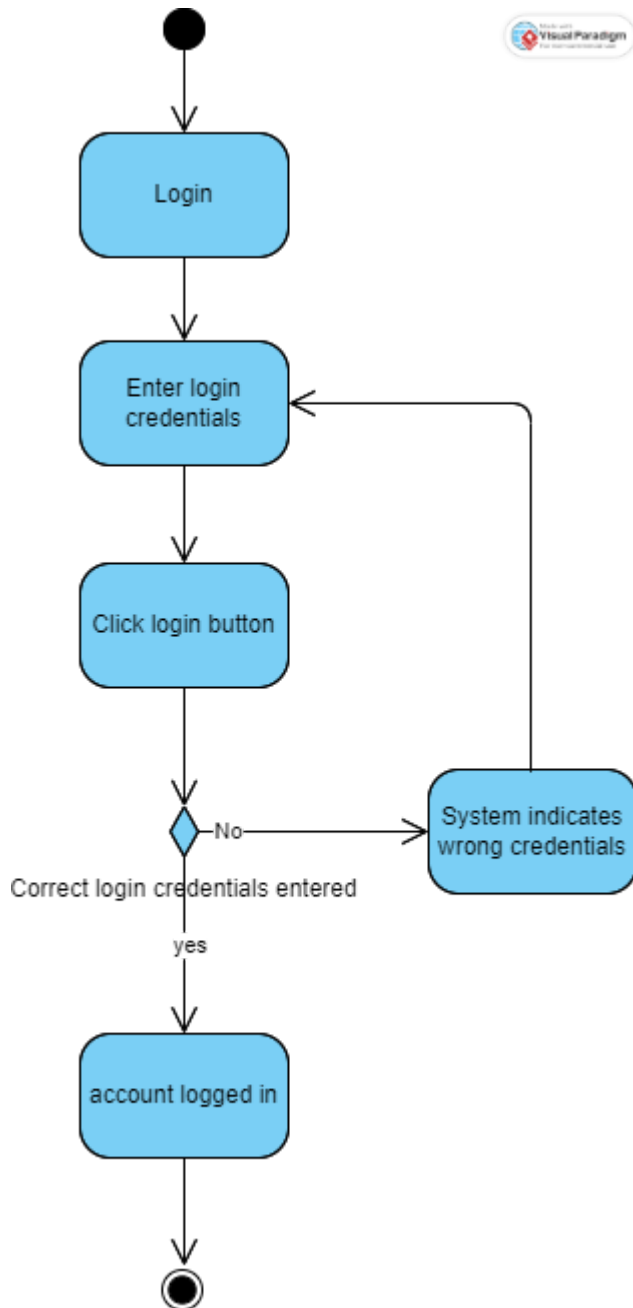


7.4. Activity Diagrams

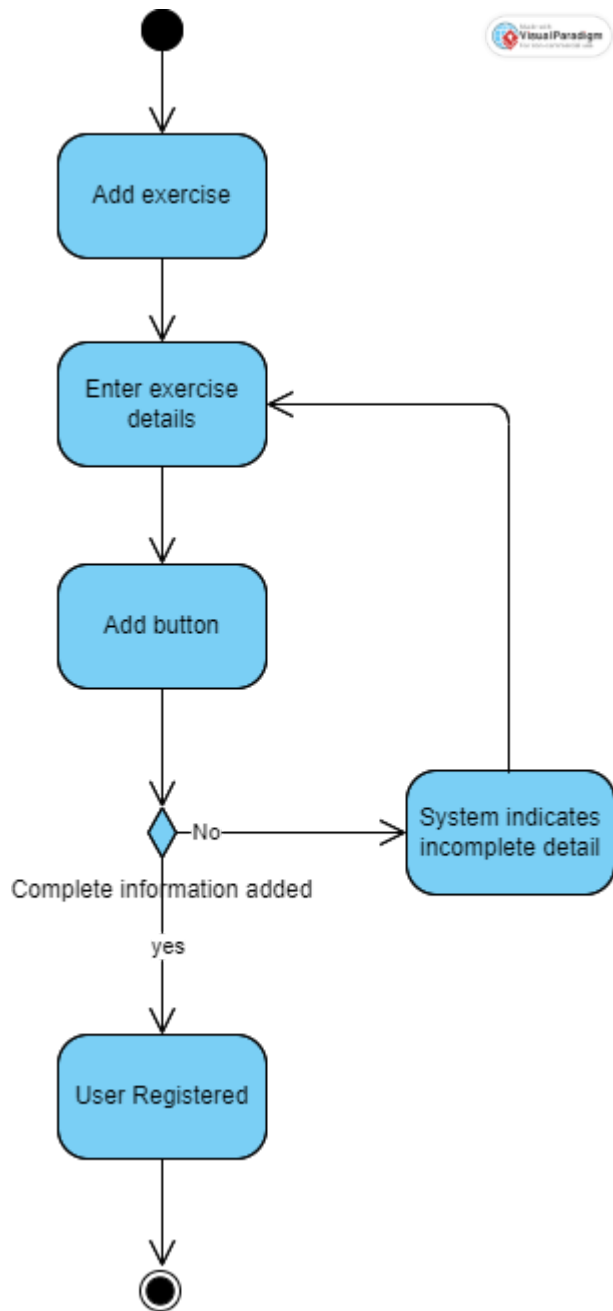
7.4.1. User Registration



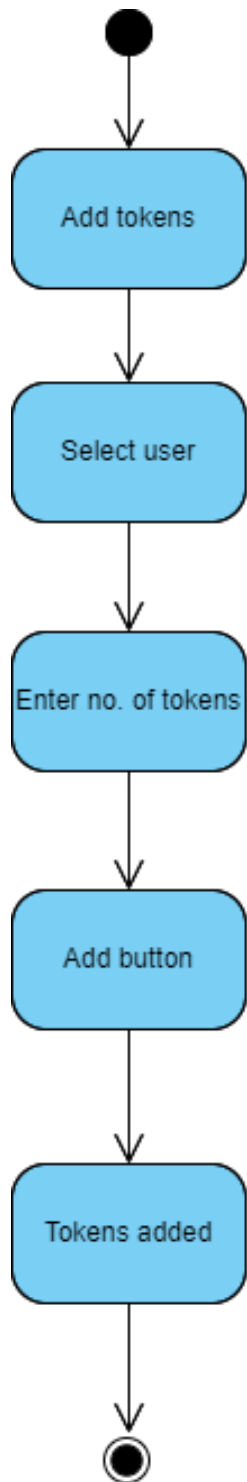
7.4.2. Login



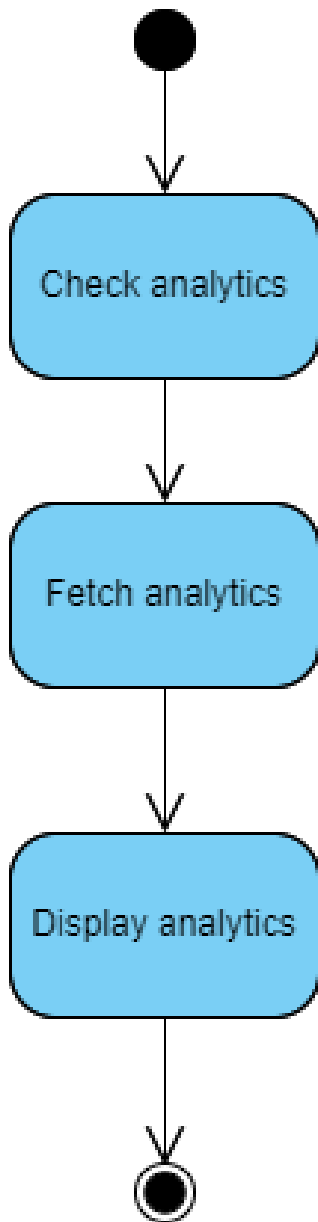
7.4.3. Add Exercise



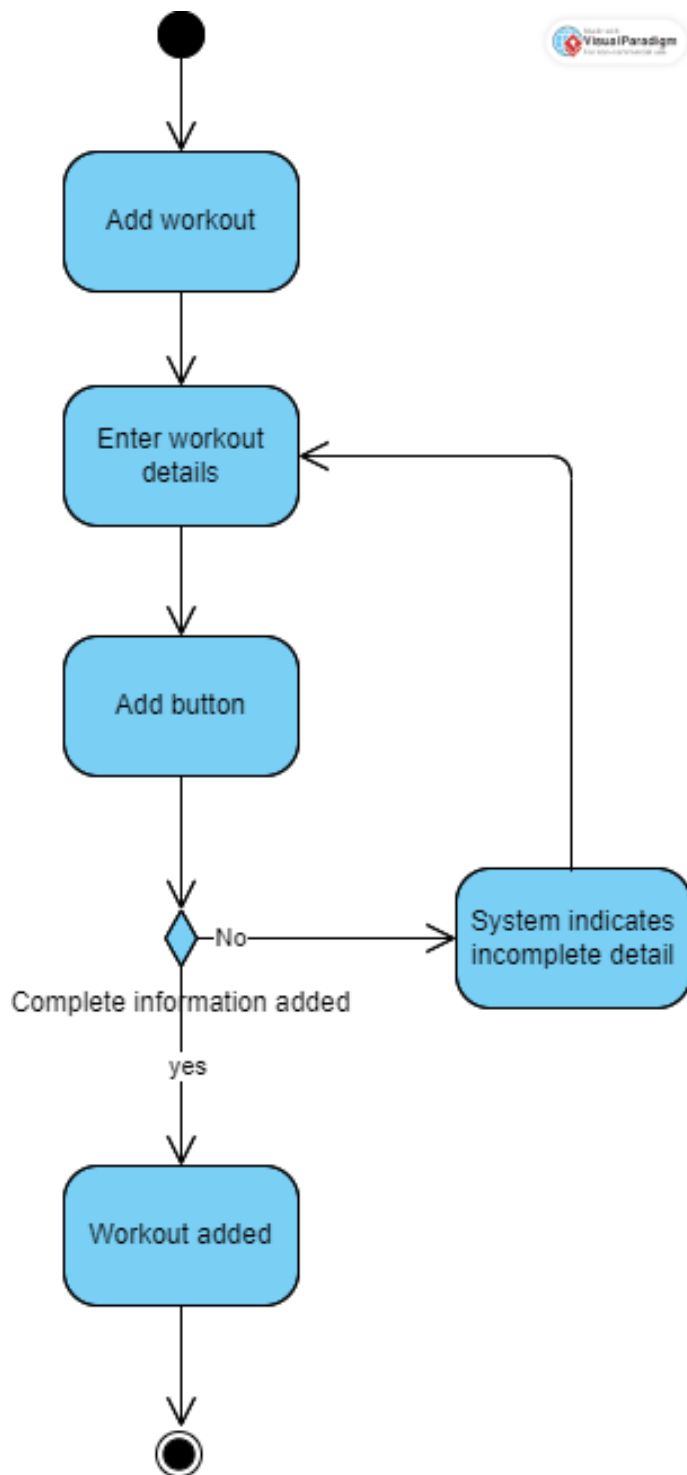
7.4.4. Add Tokens



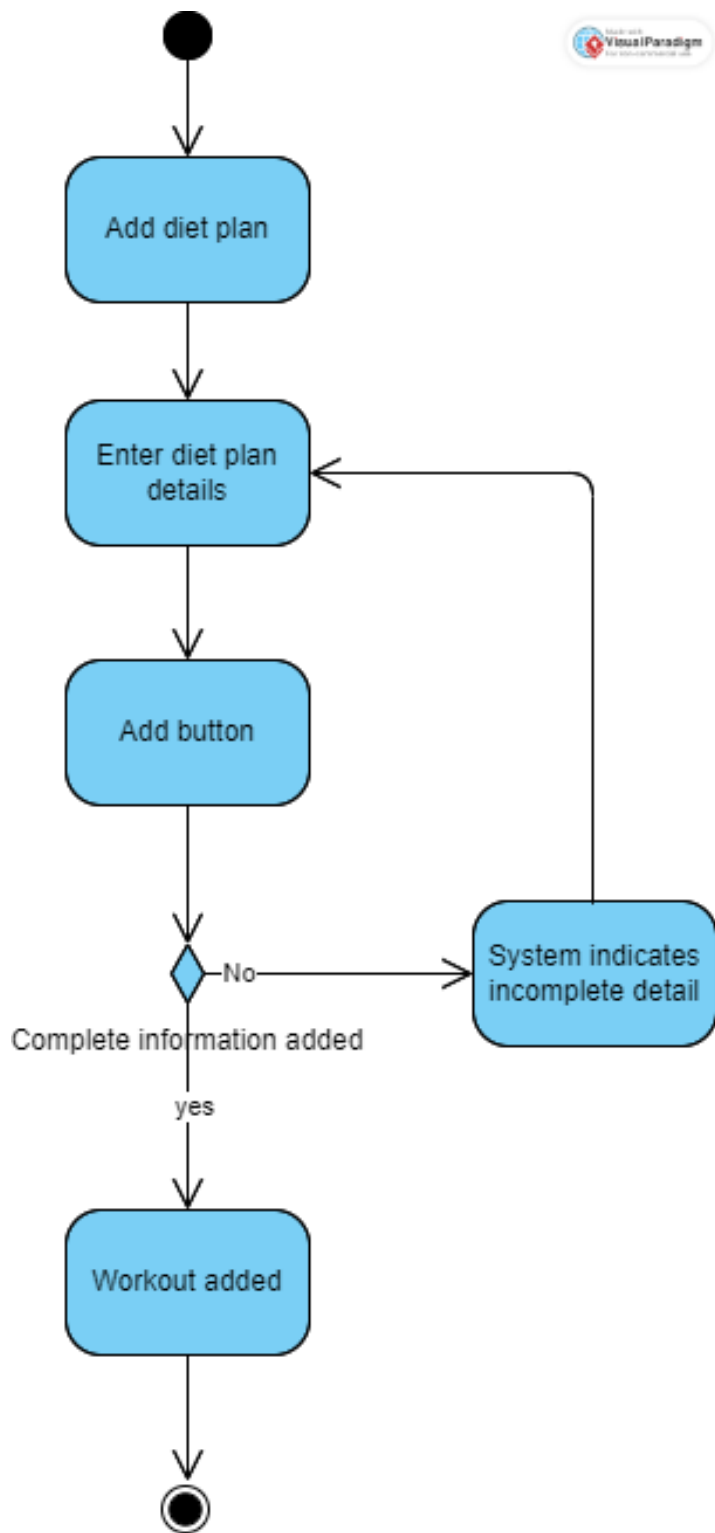
7.4.5. Check Analytics



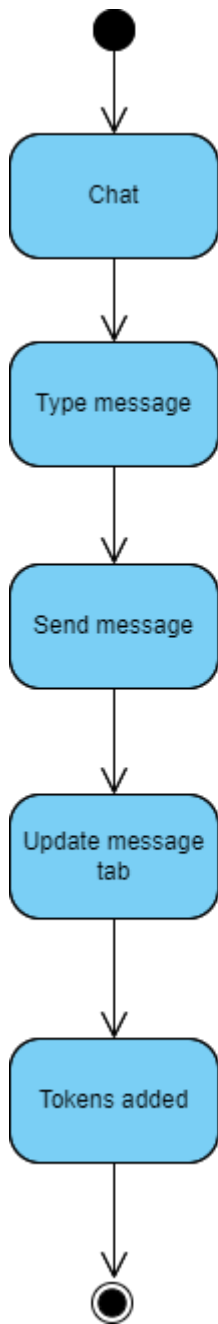
7.4.6. Add Workout



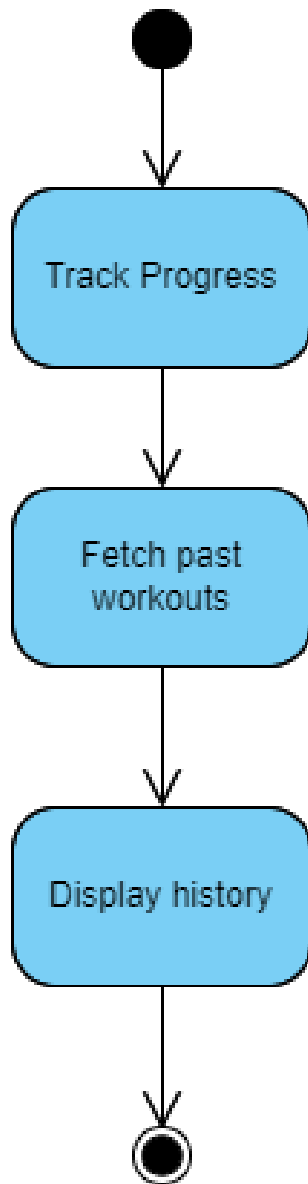
7.4.7. Add Diet Plan



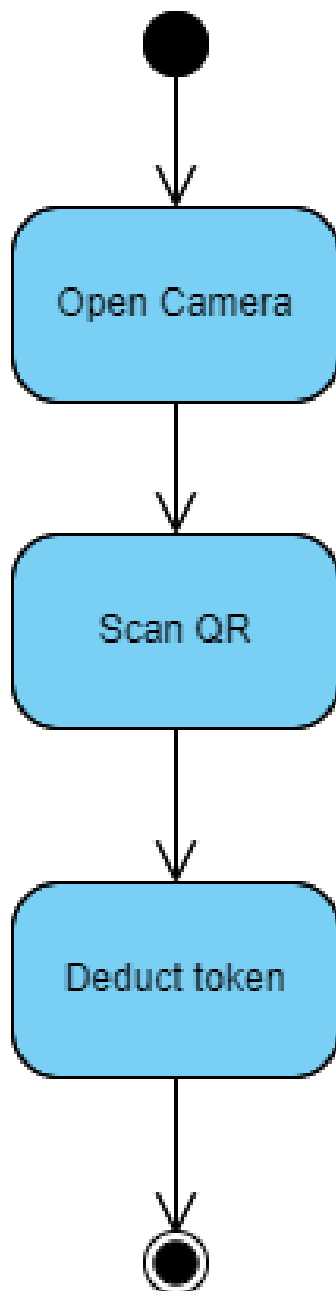
7.4.8. Chat



7.4.9. Track Progress

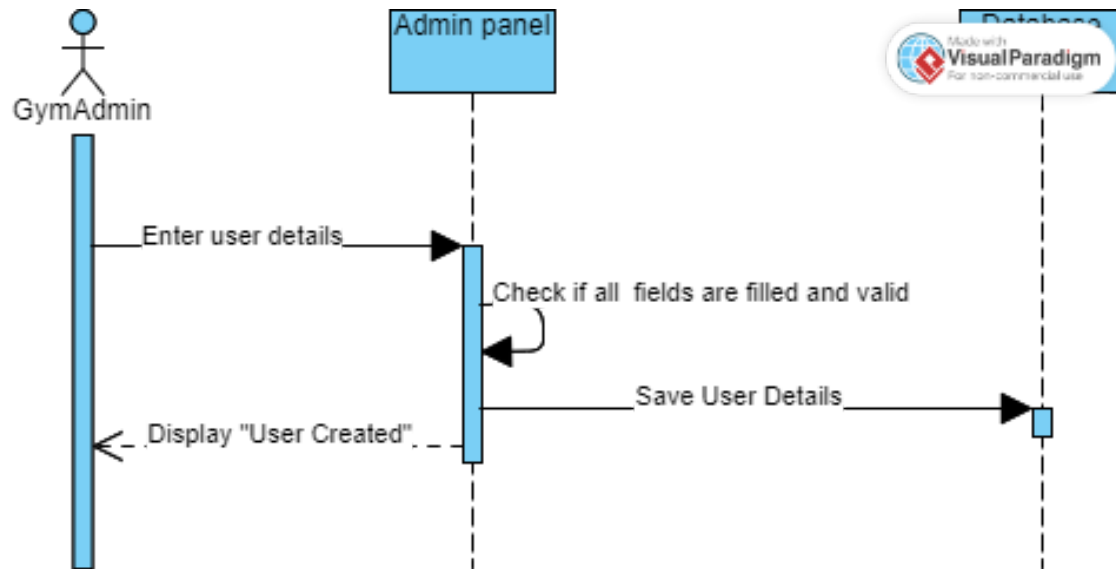


7.4.10. Scan QR

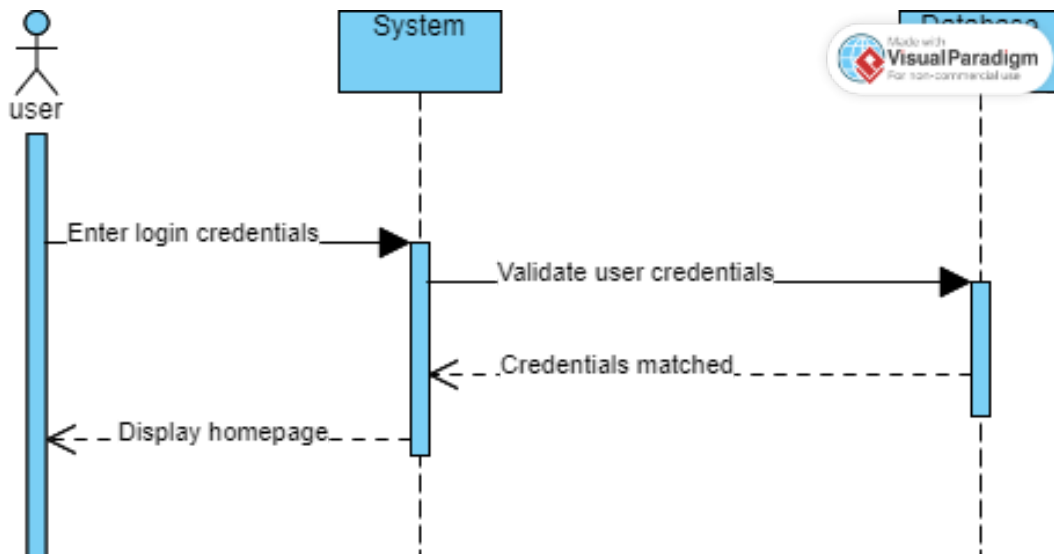


7.5. Sequence Diagrams

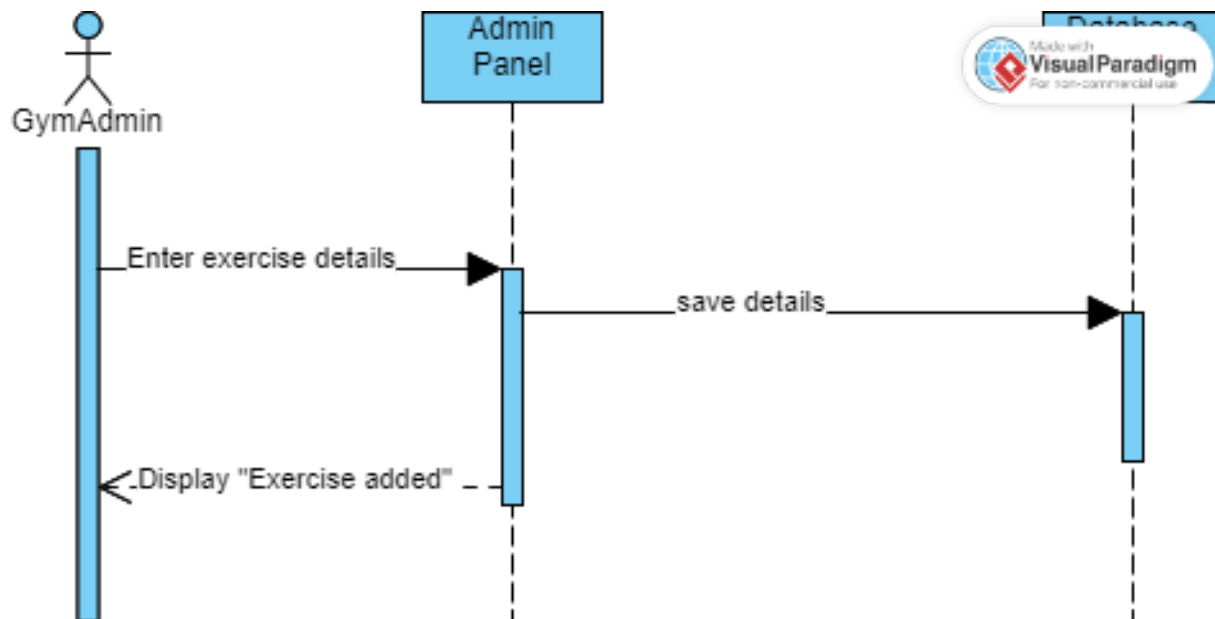
7.5.1. User Registration



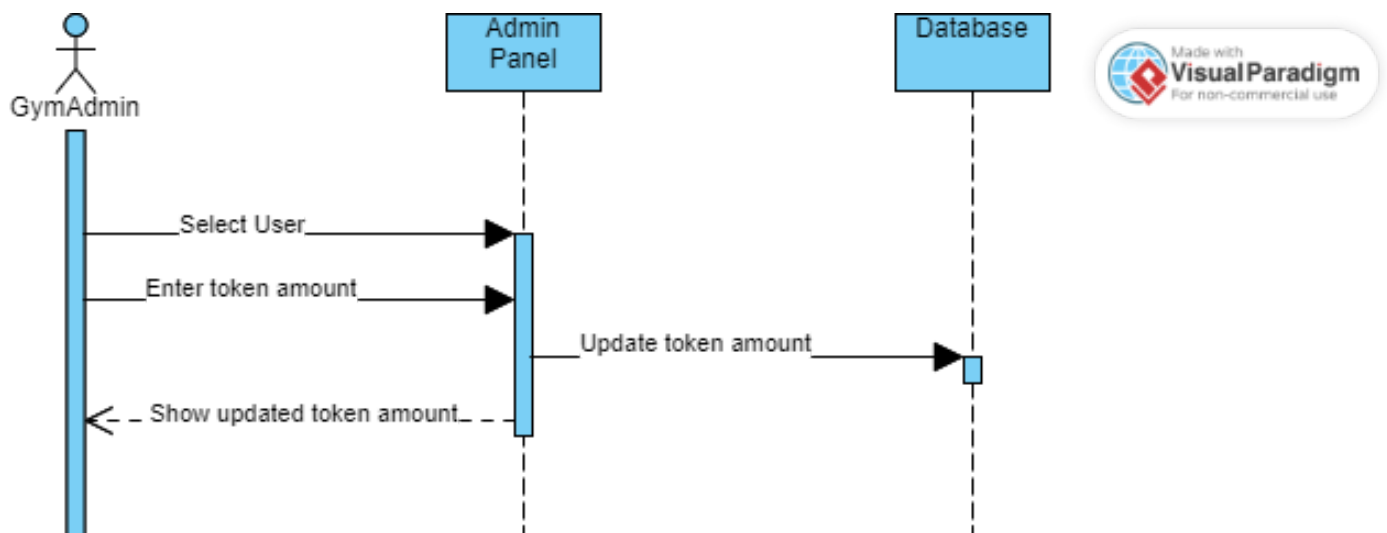
7.5.2. Login



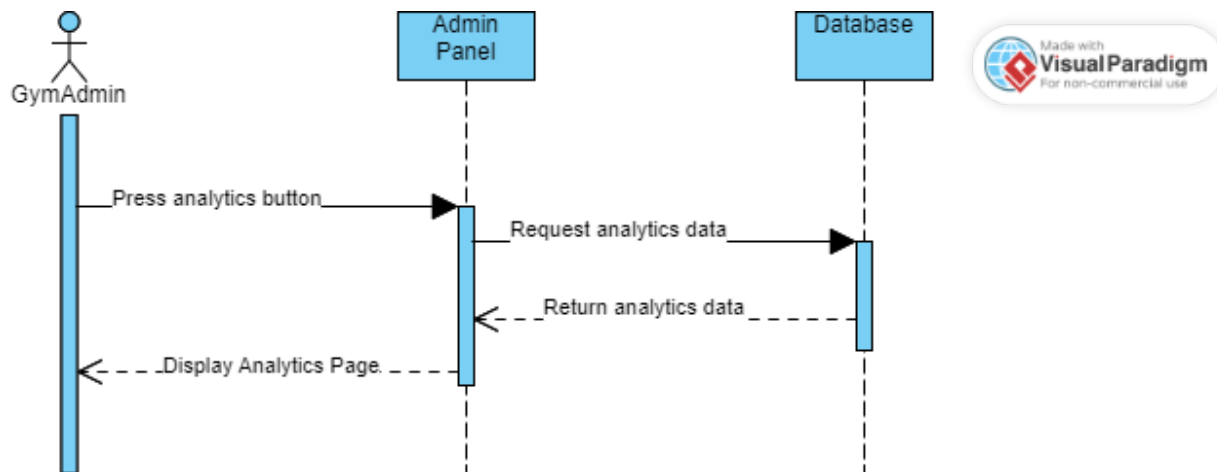
7.5.3. Add Exercise



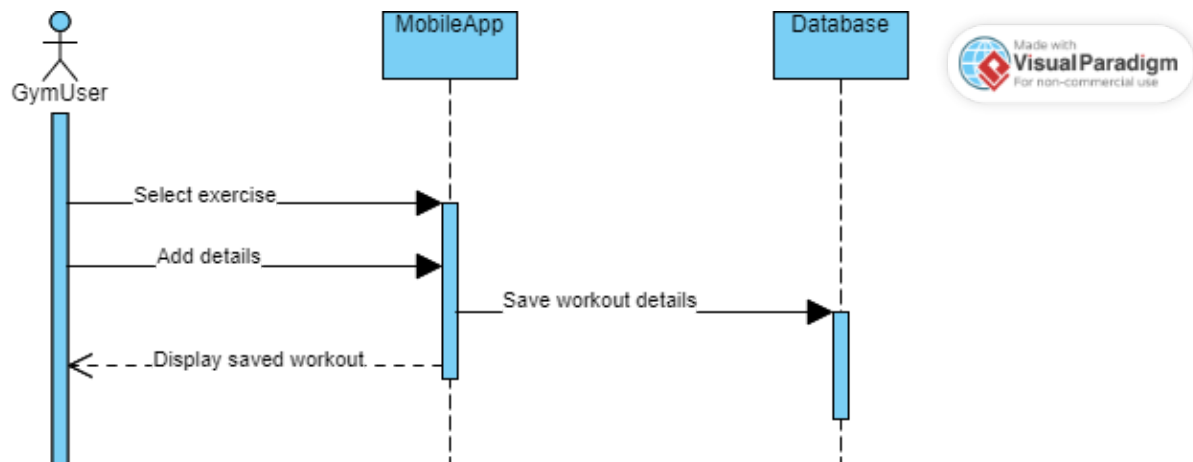
7.5.4. Add Tokens



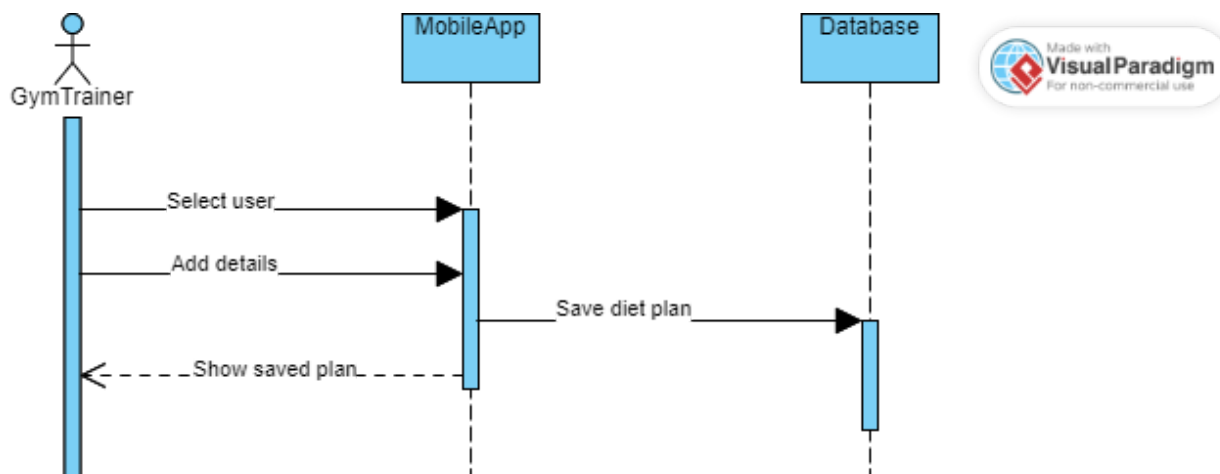
7.5.5. Check Analytics



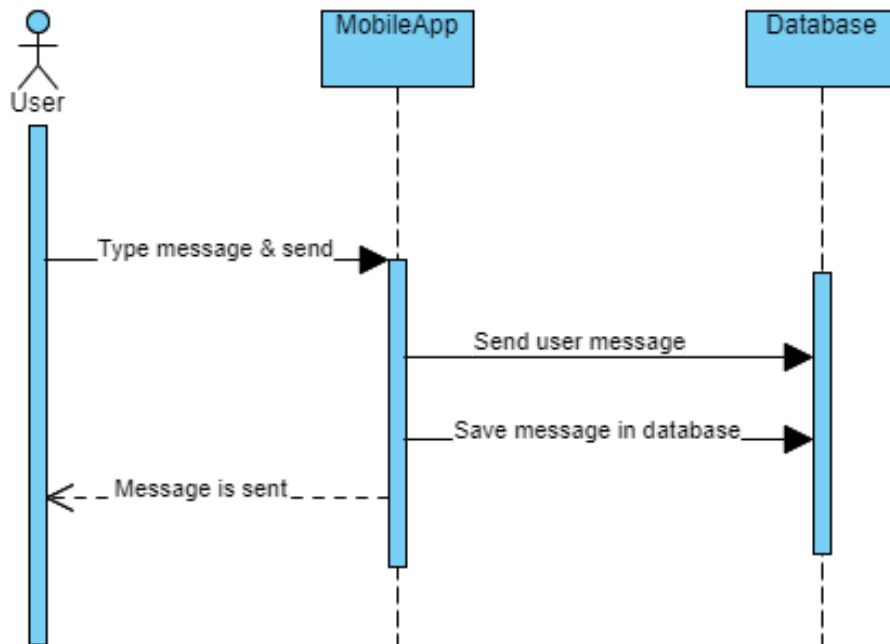
7.5.6. Add Workout



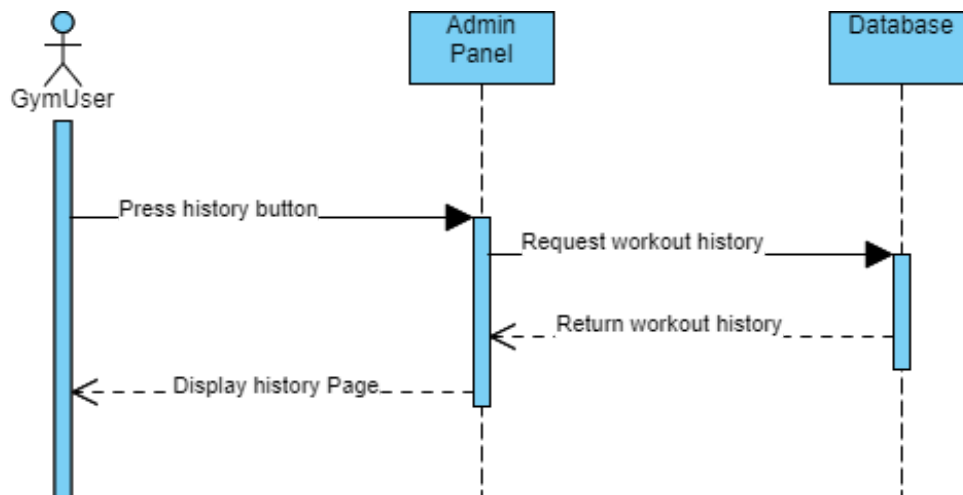
7.5.7. Add Diet Plan



7.5.8. Chat

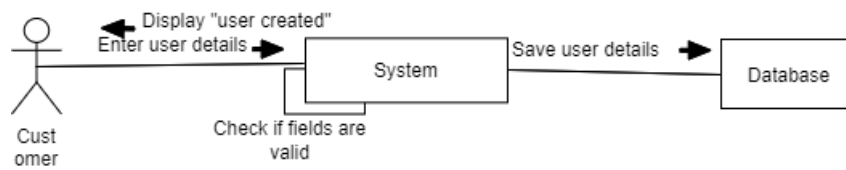


7.5.9. Track Progress

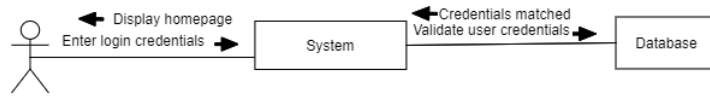


7.6. Collaboration Diagrams

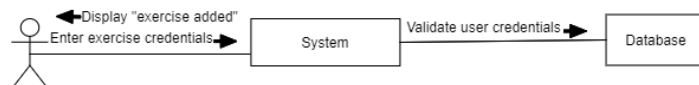
7.6.1. User Registration



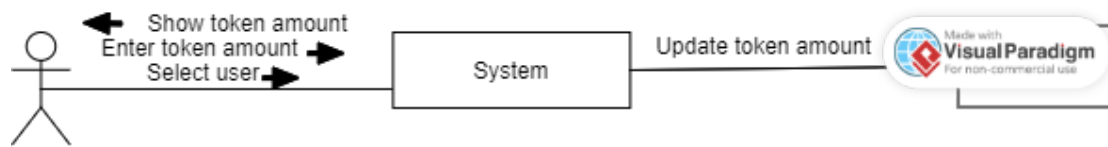
7.6.2. Login



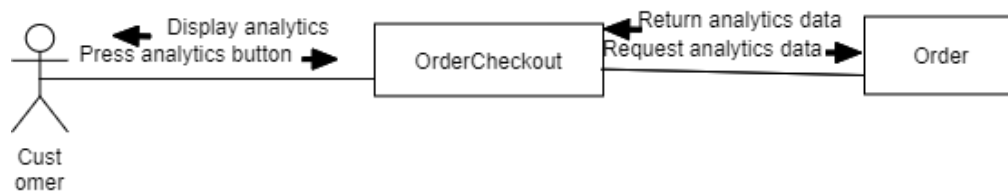
7.6.3. Add Exercise



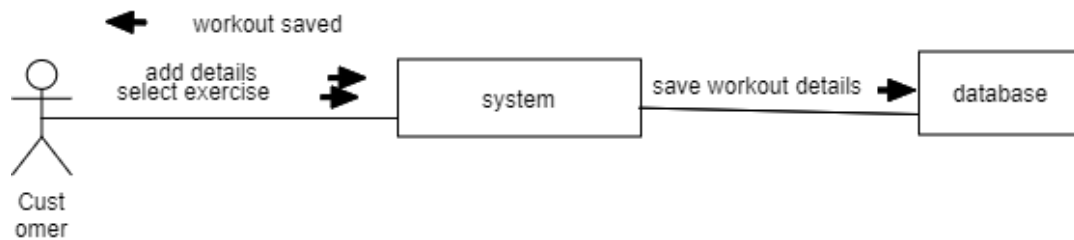
7.6.4. Add Tokens



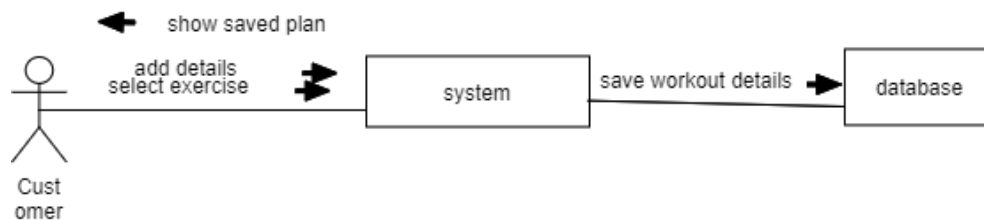
7.6.5. Check Analytics



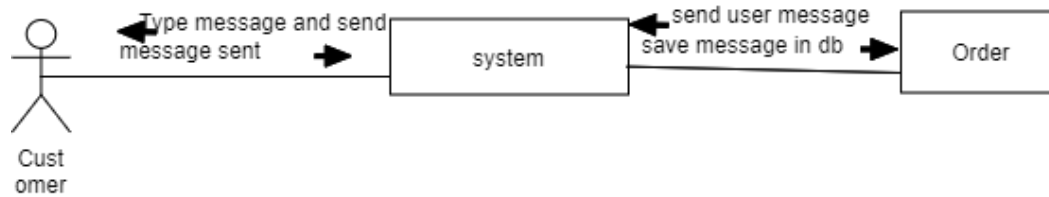
7.6.6. Add Workout



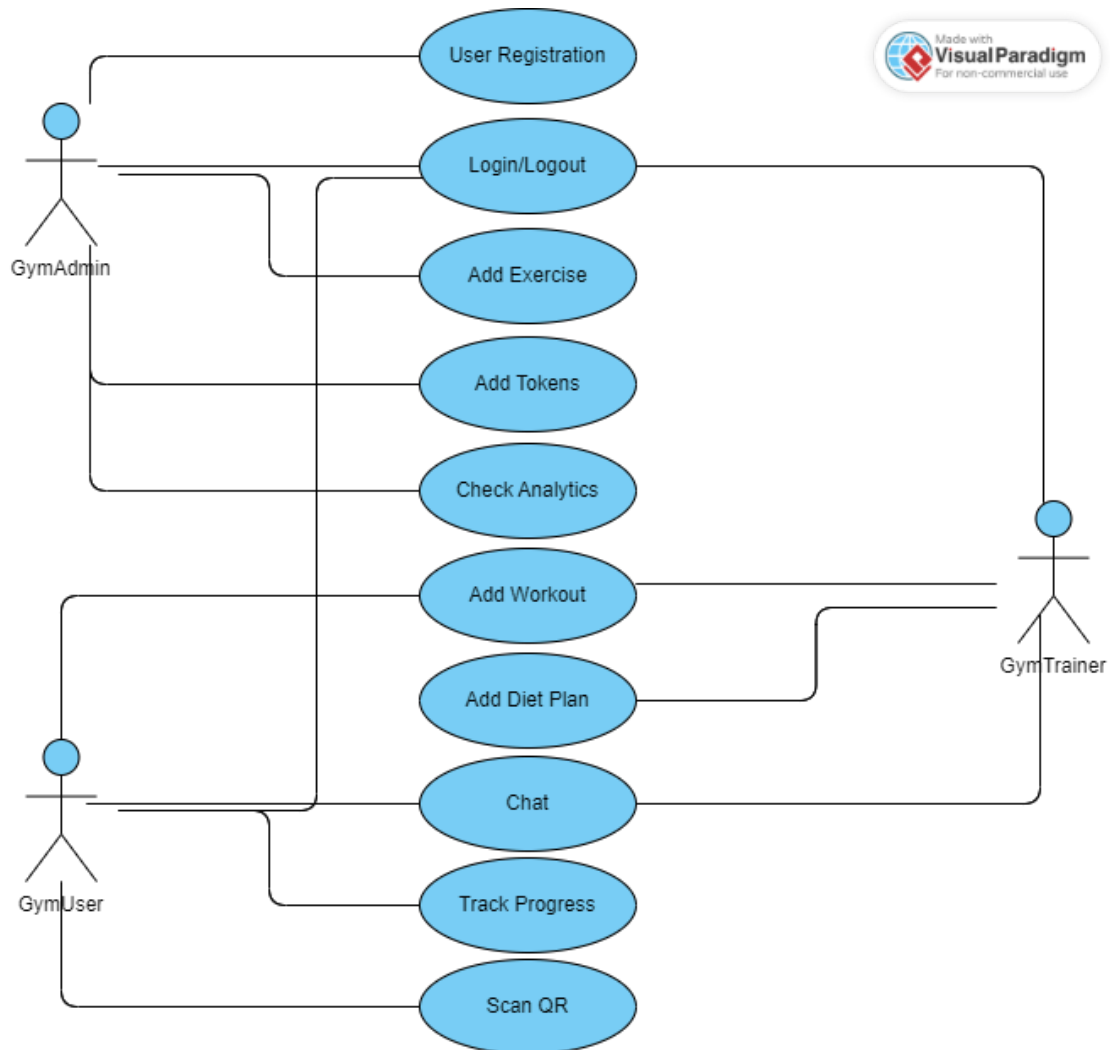
7.6.7. Add Diet Plan



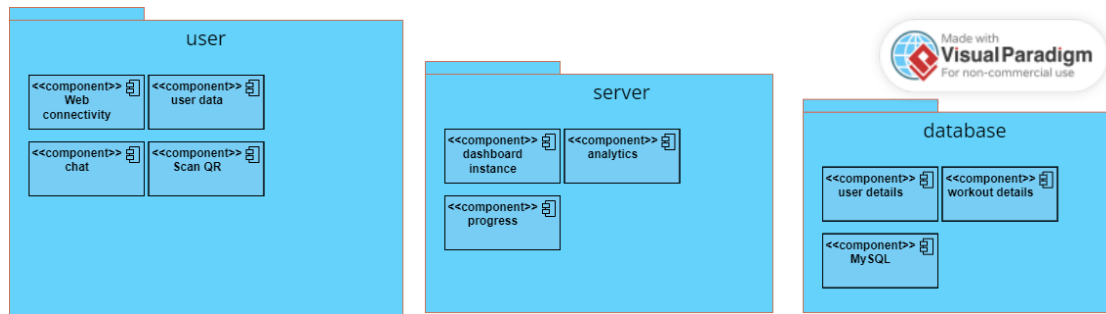
7.6.8. Chat



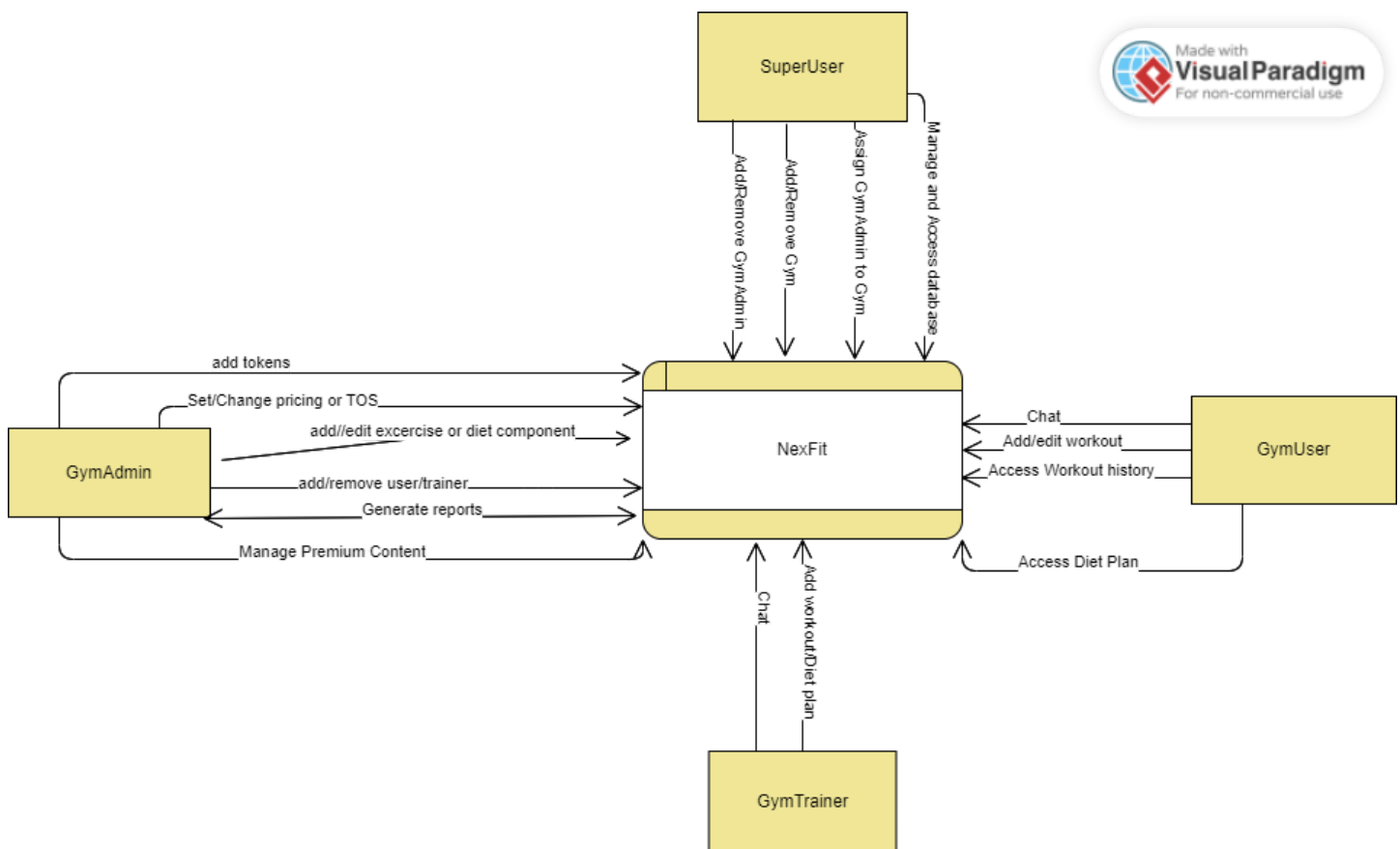
7.7. Use-Case Diagram



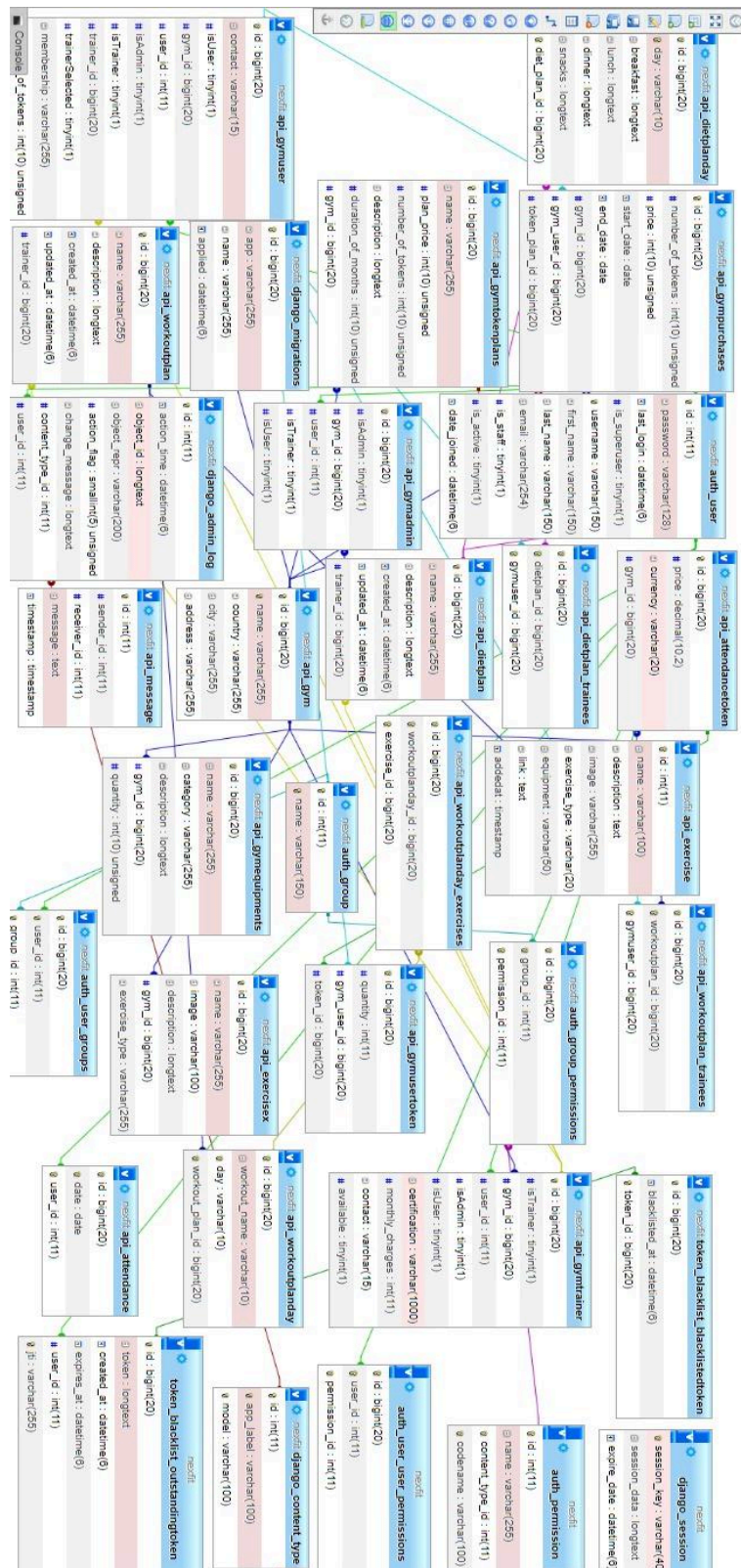
7.8. Component Diagram



7.9. State Block Diagram



68



7.11. Test Cases

7.11.1. User Registration

Test Case Name		User Registration	Test Case Description		This test case is use to test the functionality of register user feature.					
Created By		Sarim	Verion		0.1	Date Tested		Oct 2023		
S #		Prerequisites:								
1		Admin must be logged into the admin panel								
2										
3										
Step #		Step Details		Expected Results		Actual Results		Pass / Fail / Not executed / Suspended		
1		Enter text in numeric fields		System will not register input		As expected		Pass		
2		Leave both field empty and click Submit button		System will display a message " Some fields are empty "		As expected		Pass		
3		Enter Correct information		System will display a message " User Registered "		As expected		Pass		

7.11.2. Login

Test Case Name	Admin Login	Test Case Description	This test case is use to test the functionality of Login feature.					
Created By	Sarim	Verion	0.1	Date Tested	Oct 2023			
S #	Prerequisites:							
1	User must have an account							
2	Account must be active							
3								
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended			
1	Enter wrong ID and correct password	System will display a message " Wrong Credentials "	As expected		Pass			
2	Enter Correct ID and wrong Password	System will display a message " Wrong Credentials "	As expected		Pass			
3	Enter Correct ID and correct Password	Admin dashboard will be displayed	As expected		Pass			
4	Leave both field empty and click Submit button	System will display a message "Must enter ID and password"	As expected		Pass			

7.11.3. Add Exercise

Test Case Name	Add Exercise	Test Case Description	This test case is use to test the functionality of add exercise feature.					
Created By	Sarim	Verion	0.1	Date Tested	Oct 2023			
S #	Prerequisites:							
1	Admin must be logged into the admin panel							
2								
3								
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended			
1	Enter text in numeric fields	System will not register input	As expected		Pass			
2	Leave necessary field empty and click Submit button	System will display a message " Some fields are empty "	As expected		Pass			
3	Enter Correct information	System will display a message " Exercise added"	As expected		Pass			

7.11.4. Add Tokens

Test Case Name	Add Tokens	Test Case Description	This test case is use to test the functionality of Add tokens feature.					
Created By	Sarim	Verion	0.1	Date Tested	Oct 2023			
S #	Prerequisites:							
1	Admin must be logged into the admin panel							
2								
3								
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended			
1	Search user	System will list all matching results	As expected		Pass			
2	Select user	The user's details will be displayed on the screen	As expected		Pass			
3	Select token plan	Selected number of tokens will be added to the user account	As expected		Pass			

7.11.5. Check Analytics

Test Case Name	Check Analytics	Test Case Description	This test case is use to test the functionality of Check analytics feature.					
Created By	Sarim	Verion	0.1	Date Tested	Oct 2023			
S #	Prerequisites:							
1	Admin must be logged into the admin panel							
2								
3								
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended			
1	Click the analytics button	System will fetch analytics data from database and display it	As expected		Pass			

7.11.6. User Login

Test Case Name	User Login	Test Case Description	This test case is use to test the functionality of user login feature.					
Created By	Sarim	Verion	0.1	Date Tested	Oct 2023			
S #	Prerequisites:							
1	User must be connected to the internet							
2								
3								
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended			
1	Enter text in numeric fields	System will not register input	As expected		Pass			
2	Leave both field empty and click Submit button	System will display a message " Some fields are empty "	As expected		Pass			
3	Enter Correct information	System will redirect to homepage	As expected		Pass			

7.11.7. User Profile

Test Case Name	User Profile	Test Case Description	This test case is use to test the functionality of user profile feature.					
Created By	Sarim	Verion	0.1	Date Tested	Oct 2023			
S #	Prerequisites:							
1	User must be connected to the internet							
2								
3								
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended			
1	Click the profile button	System will fetch user details from db and display it	As expected		Pass			

7.11.8. Exercises

Test Case Name	Exercises	Test Case Description	This test case is use to test the functionality of Exercises feature.					
Created By	Sarim	Verion	0.1	Date Tested	Oct 2023			
S #	Prerequisites:							
1	User must be connected to the internet							
2								
3								
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended			
1	Click the exercises button	System will fetch exercises data for that gym and display it	As expected		Pass			

7.11.9. Scan QR

Test Case Name	Scan QR	Test Case Description	This test case is use to test the functionality of Scan QR feature.					
Created By	Sarim	Verion	0.1	Date Tested	Oct 2023			
S #	Prerequisites:							
1	User must be connected to the internet							
2								
3								
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended			
1	Click QR button	System will show attendance	As expected		Pass			
2	Click camera button	System will open camera	As expected		Pass			
3	Scan a random QR code	System will not do anything and stay on the camera	As expected		Pass			
4	Scan the correct QR code	System will deduct a token and return to home page	As expected		Pass			

7.11.10. Chat

Test Case Name	Chat	Test Case Description	This test case is use to test the functionality of chat feature.					
Created By	Sarim	Verion	0.1	Date Tested	Oct 2023			
S #	Prerequisites:							
1	User must be connected to the internet							
2								
3								
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended			
1	Type text in message box	System will display text in text box	As expected		Pass			
2	Click send	System will send the message and update screen	As expected		Pass			

7.11.11. Diet Plan

Test Case Name	Add Diet Plan	Test Case Description	This test case is use to test the functionality of add diet plan feature.					
Created By	Sarim	Verion	0.1	Date Tested	Oct 2023			
S #	Prerequisites:							
1	Trainer must be logged in							
2								
3								
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended			
1	Click add diet plan button	System will redirect to diet plan form	As expected		Pass			
2	Assign diet plan to trainee	System will associate the diet plan to the specified user	As expected		Pass			
3	Submit the form	System will save the changes to the user and trainee(s) profiles	As expected		Pass			

7.11.12. Workout Plan

Test Case Name	Add Workout Plan	Test Case Description	This test case is use to test the functionality of add workout plan feature.					
Created By	Sarim	Verion	0.1	Date Tested	Oct 2023			
S #	Prerequisites:							
1	Trainer must be logged in							
2								
3								
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended			
1	Click add Workout plan button	System will redirect to workout plan form	As expected		Pass			
2	Assign Workout plan to trainee	System will associate the workout plan to the specified user	As expected		Pass			
3	Submit the form	System will save the changes to the user and trainee(s) profiles	As expected		Pass			